ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО
Научный руководитель,
доцент департамента
программной инженерии

_____Р.А. Нестеров
«__» _____2026 г.

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»
Павлочев Николай Александрович

_____Н.А.Павлочев
«__» _____2026 г.

Выпускная квалификационная работа
(проектная)

на тему:

Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть

по направлению подготовки 09.03.04 «Программная инженерия»

ВЫПОЛНИЛ
студент группы БПИ 228
образовательной программы
09.03.04 «Программная инженерия»

Д.Ю.Маслов

__.__.2026
Подпись, Дата

Москва 2026

Реферат

В современном мире поддержание здорового образа жизни требует систематического подхода к физическим нагрузкам и питанию, что зачастую представляет сложность из-за необходимости индивидуального планирования и контроля. Для решения этой проблемы разрабатывается серверная часть интеллектуального фитнес-приложения для персонализированного контроля питания и тренировок с использованием технологий искусственного интеллекта и интеграцией данных о здоровье пользователя. Система реализует автоматическую генерацию адаптивных тренировочных программ на основе анализа физических параметров пользователя, его целей и истории активности, а также интеллектуальный анализ питания с распознаванием блюд по фотографиям через Open AI API. Платформа включает механизмы прогрессивной нагрузки, динамической корректировки планов, расчет суточных норм калорийности и персонализированных рекомендаций. Дополнительно реализованы система уведомлений через различные каналы связи (push, SMS, email) и управляемый процесс регистрации с JWT-аутентификацией. Разрабатываемое решение использует нейросетевые модели для обработки пользовательских данных и анализа визуального контента, применяя научно обоснованные методики тренировочного процесса для оптимизации физического развития пользователей.

Работа содержит ? страниц, 3 главы, ? иллюстрации и ? источника.

Ключевые слова: *фитнес-приложение; искусственный интеллект; анализ питания; персонализированные тренировки; JWT-аутентификация; PostgreSQL; REST API.*

Abstract

In the modern world, maintaining a healthy lifestyle requires a systematic approach to physical activity and nutrition, which often presents challenges due to the need for individual planning and monitoring. To address this problem, the server-side of an intelligent fitness application for personalized nutrition and workout control using artificial intelligence technologies and integration of user health data is being developed. The system implements automatic generation of adaptive training programs based on analysis of user's physical parameters, goals, and activity history, as well as intelligent nutrition analysis with food recognition from photos via OpenAI API. The platform includes mechanisms for progressive overload, dynamic plan adjustments, daily calorie norm calculations, and personalized recommendations. Additionally, a notification system through various communication channels (push, SMS, email) and a managed registration process with JWT authentication are implemented. The developed solution utilizes neural network models for processing user data and analyzing visual content, applying scientifically based training methodologies to optimize users' physical development.

The work contains ? pages, 3 chapters, ? figures and ? sources.

Keywords: fitness application; artificial intelligence; nutrition analysis; personalized workouts; JWT authentication; PostgreSQL; REST API.

Содержание

Основные определения, термины и сокращения 6
Введение 8
Глава 1. Предметная область и существующие решения 11
1.1. Описание предметной области и актуальность 11
1.2. Общее описание сервисов 12
1.3. MyFitnessPal 14
1.4. FatSecret 15
1.5. Strong 17
1.6. Яндекс.Здоровье 18
1.7. 8fit 20
1.8. Сравнительный анализ 21
Выводы по главе 25
Глава 2. Проектирование системы и технологии разработки 26
2.1. Основной функционал сервисов 26
2.2. Пользовательские сценарии 28
2.2.1. Ключевые пользовательские сценарии 28
2.2.2. Диаграммы прецендентов 30
2.2.3. Валидация полноты реализации 32
2.3. Архитектура решения 33
2.3.1. Обоснование выбора архитектуры 33
2.4. Проектирование сервиса 36
2.4.1. Общая архитектура системы 37
2.5. Проектирование базы данных 39
2.5.1. Обоснование выбора PostgreSQL и стратегии проектирования 39
2.5.2. Общая структура схемы данных 39
2.5.3. Модель пользователей и профиля 40
2.5.4. Модель тренировок и упражнений 42
2.5.5. Модель питания и мониторинга 43
2.5.6. Модель фоновых задач 44
2.5.7. Миграции и управление схемой 45
Выводы по главе 46
Глава 3. Программная реализация серверной части 47
3.1. Структура проекта и организация кода 47
3.2. Реализация модуля аутентификации (Auth Service) 48
3.3. Реализация модуля управления данными (CRUD Service) 49
3.4. Реализация модуля тренировок (Workout Service) 50
3.4.1. Фоновая генерация тренировок 51
3.4.2. Анализ необходимости генерации 51
3.4.3. Процесс генерации тренировки 52
3.4.4. Генерация тренировки по запросу 53
3.4.5. Обработка ошибок и отказоустойчивость 54
3.5. Реализация модуля питания (Nutricion Service) 54
3.6. Реализация модуля рекомендаций (Recommendation Service) 56
3.7. Реализация модуля уведомлений (Executor Service) 57
3.8. Реализация модуля аватаров (Avatar Service) 58

Выводы по главе 59
Заключение 61
Список использованных источников 63

Основные определения, термины и сокращения

1. **Монолитная архитектура** - архитектурный стиль, при котором все компоненты системы развёртываются как единое целое, выбранный для проекта данного сервиса.
2. **Clean Architecture (Чистая архитектура)** - подход с разделением на слои (domain, application, infrastructure, delivery), использованный для организации кода
3. **DDD (Domain-Driven Design)** - методология проектирования, основанная на выделении доменных моделей (User, Workout, Nutrition, Recommendation).
4. **JWT (JSON Web Tokens)** - стандарт аутентификации с использованием access и refresh токенов.
5. **BCrypt** - алгоритм хеширования паролей, используемый для безопасного хранения учётных данных.
6. **OpenAI API** – апи нейросети
7. **Executor Service** - сервис отложенных задач для обработки уведомлений (push, SMS, email).
8. **S3-совместимое хранилище (MinIO)** - объектное хранилище для изображений (аватары, фото еды).
9. **TDEE (Total Daily Energy Expenditure)** - суточный расход энергии, рассчитываемый по формуле Миффлина-Сан Жеора.
10. **Lifestyle (Образ жизни)** - динамический параметр пользователя (not_active/active/sportive), меняющийся на основе активности.
11. **LevelPreparation (Уровень подготовки)** - показатель физической готовности (beginner/medium/sportsman).
12. **Workout Status (Статус тренировки)** - состояние тренировки: created, in_active, done, failed.
13. **API — это набор правил и протоколов**, который позволяет различным программным компонентам взаимодействовать друг с другом.
14. **Exercise Type (Тип упражнения)** - категоризация упражнений по мышечным группам.
15. **Place Exercise (Место тренировки)** - категория места занятий: home, gym, street.
16. **КБЖУ** - калории, белки, жиры, углеводы; определяемые через анализ питания.
17. **REST API** - архитектурный стиль API, используемый для взаимодействия с клиентом.
18. **PostgreSQL** - реляционная СУБД для хранения всех данных системы.
19. **Redis** - хранилище "ключ-значение" для кэширования данных.
20. **CRUD** - базовые операции Create, Read, Update, Delete для управления данными.
21. **Фоновый сервис** - компонент, выполняющий задачи без прямого запроса пользователя (генерация тренировок).
22. **Push-уведомления** - сообщения, отправляемые на мобильное устройство пользователя.
23. **Промпт (Prompt)** - запрос к OpenAI API для анализа изображений.
24. **Валидация** - проверка корректности входных данных.
25. **Docker** - технология контейнеризации для развёртывания приложения.
26. **CI/CD** - практики непрерывной интеграции и доставки.
27. **Prometheus** - система мониторинга метрик производительности.

28. **Горутина** – легковесный поток выполнения в языке Go, который позволяет конкурентно выполнять функции.
29. **Эндпоинт** – конечная точка API, определяющая конкретный URL и HTTP-метод, по которым клиент может взаимодействовать с сервером.
30. **Промпт -** текстовый запрос, отправляемый к нейросети для получения структурированного ответа.
31. **Метрики** - показатели работы системы (время ответа, частота ошибок).
32. **Генерация тренировок** - автоматическое создание персонализированных программ упражнений.

Введение

В условиях стремительной цифровизации сферы здравоохранения и фитнеса, а также роста осознанности населения в вопросах здорового образа жизни, всё большую актуальность приобретают интеллектуальные системы, способные персонализировать подход к физической активности и питанию. Современные пользователи сталкиваются с необходимостью самостоятельного планирования тренировочных программ, расчёта нутриентов и отслеживания прогресса, что требует значительных временных затрат и специальных знаний. Существующие фитнес-приложения зачастую предлагают стандартизированные решения без учёта индивидуальных особенностей пользователей, их целей, текущей физической формы и предпочтений.

Особенно остро проблема отсутствия персонализации проявляется для людей, стремящихся к систематическому улучшению физической формы без возможности обращения к персональным тренерам и диетологам. Пользователи вынуждены самостоятельно изучать сложные методики тренировок, рассчитывать нормы потребления калорий, анализировать пищевую ценность продуктов и отслеживать прогресс, что делает процесс достижения фитнес-целей трудоёмким и зачастую неэффективным. Более того, традиционные приложения не адаптируются к изменениям состояния пользователя и не учитывают его прогресс в динамике.

Поэтому в рамках данной выпускной квалификационной работы разрабатывается серверная часть интеллектуальной фитнес-платформы — высоконагруженная система для персонализированного контроля питания и тренировок с использованием технологий искусственного интеллекта. Система реализует автоматическую генерацию адаптивных тренировочных программ на основе анализа физических параметров пользователя, его целей и истории активности, а также интеллектуальный анализ питания с распознаванием блюд по фотографиям через интеграцию с OpenAI API. Платформа динамически корректирует планы тренировок, применяя принципы прогрессивной нагрузки, и рассчитывает индивидуальные суточные нормы калорийности.

Серверная часть системы состоит из нескольких взаимосвязанных компонентов:

1. Сервис аутентификации (Auth Service) – отвечает за регистрацию, авторизацию пользователей с использованием JWT-токенов, верификацию email и телефона, восстановление доступа к аккаунту и управление refresh-токенами;
2. Сервис тренировок (Workout Service) – реализует автоматическую генерацию персонализированных тренировочных программ на основе целей пользователя, уровня

подготовки, истории активности и доступного оборудования, применяет принцип прогрессивной нагрузки и динамической адаптации сложности;

3. Сервис питания (Nutrition Service) – обеспечивает анализ фотографий еды через Open AI для определения пищевой ценности (КБЖУ), ведение дневника питания, расчёт суточного потребления калорий и нутриентов, сравнение с целевыми показателями;
4. Сервис рекомендаций (Recommendation Service) – генерирует персонализированные ежедневные рекомендации по питанию, тренировкам и восстановлению на основе анализа прогресса пользователя и интеграции с Open AI для расширенной текстовой генерации советов;
5. Сервис отложенных задач (Executor Service) – управляет очередью задач на отправку уведомлений (push, SMS, email), обеспечивает повторные попытки при неудачной доставке, интегрируется с внешними провайдерами;
6. CRUD-сервис – предоставляет базовые операции создания, чтения, обновления и удаления для всех сущностей системы (упражнения, пользовательские данные, пищевые записи, задачи и др.);
7. Сервис аватаров (Avatar Service) – управляет загрузкой, обновлением и удалением пользовательских фотографий в S3-совместимом хранилище.

Цель разработки *выпускной квалификационной работы (ВКР) – разработать серверную часть интеллектуальной фитнес-платформы, обеспечивающую автоматическую генерацию персонализированных тренировочных программ и интеллектуальный анализ питания с использованием технологий искусственного интеллекта.*

Задачи, поставленные для реализации разработки серверной части:

1. Анализ предметной области и существующих решений в сфере фитнес-приложений.
2. Выбор аналогов и проведение сравнительного анализа функциональности.
3. Определение требований к системе и разработка технического задания.
4. Проектирование архитектуры системы с использованием принципов Clean Architecture и DDD.
5. Разработка схемы базы данных и реализация моделей данных.
6. Реализация сервиса аутентификации с JWT и механизмом refresh-токенов.
7. Разработка сервиса тренировок с алгоритмами автоматической генерации программ.
8. Создание сервиса питания с интеграцией Open AI API для анализа изображений.
9. Реализация сервиса рекомендаций с генерацией персонализированных советов.
10. Разработка сервиса отложенных задач для управления уведомлениями.
11. Интеграция всех сервисов в единую систему и обеспечение их взаимодействия.

12. Тестирование функциональности системы и проверка корректности работы.
13. Разработка программной документации в соответствии с требованиями ГОСТ.

Выпускная квалификационная работа состоит из трех глав.

В **первой главе** обсуждается предметная область. Также в этой главе проводится анализ функциональности ближайших сервисов-конкурентов, которые обладают функциональностью схожей с разрабатываемой. Эти сервисы сравниваются по главным критериям согласно сравнительному анализу, делается вывод об актуальности темы ВКР.

Во **второй главе** приводится выбор и обоснование методов и подходов в разработке, а также задействованных технических и программных средств. Приводится разработанная архитектура приложения с обоснованием паттерна проектирования. Подробно рассматривается устройство всего бэкенда.

В **третьей главе** содержится описание основного функционала системы, приведены и обоснованы подходы к разработке приложения.

В **заключении** подводятся итоги работы

Глава 1. Предметная область и существующие решения

1.1. Описание предметной области и актуальность

Описание области исследования и значимость проекта

Разрабатываемая система представляет собой интеллектуальную серверную платформу для персонализированного управления здоровьем, физической активностью и питанием, реализующую комплексный подход к формированию здорового образа жизни с применением технологий искусственного интеллекта. Платформа функционирует как централизованный процессор, обеспечивающий хранение, обработку и анализ мультимодальных пользовательских данных для генерации индивидуальных фитнес-программ, рекомендаций по питанию и мотивационного сопровождения.

В рамках проекта реализуется функциональность, связанная с автоматизированной генерацией тренировочных программ на основе анализа физиологических параметров пользователя, его целей, уровня подготовки и истории активности. Система применяет принципы спортивной физиологии и методологию прогрессивной нагрузки для создания адаптивных планов, динамически корректирующихся в соответствии с достигнутым прогрессом. Особое внимание уделяется алгоритмам учёта доступного оборудования и места тренировок (домашние условия, тренажёрный зал, уличные тренировки), обеспечивающим практическую реализуемость рекомендаций.

Одним из основных компонентов является модуль интеллектуального анализа питания, использующий интеграцию с Open AI API для распознавания блюд по фотографиям и автоматического определения их пищевой ценности. Платформа обеспечивает ведение детализированного дневника питания с расчётом суточного баланса калорий и нутриентов, сравнением фактического потребления с целевыми показателями, а также формированием аналитических отчётов.

Система поддерживает многофакторное взаимодействие с пользователем через реализацию сервиса персонализированных рекомендаций, генерирующего ежедневные советы по тренировкам, питанию и восстановлению на основе анализа данных пользователя и его прогресса. Механизмы вовлечения пользователя реализованы через сервис отложенных задач, обеспечивающий отправку контекстных уведомлений через multiple каналы коммуникации (push-уведомления, SMS, email) с поддержкой повторных попыток доставки и логированием статуса.

Инфраструктурная часть платформы построена на принципах Clean Architecture и Domain-Driven Design (DDD) с чётким разделением на доменные контексты (User, Workout, Nutrition, Recommendation) и архитектурные слои (domain, application, infrastructure, delivery). Система использует централизованное хранение данных в PostgreSQL, кэширование в Redis, объектное хранилище для медиафайлов (S3/MinIO) и обеспечивает безопасность через JWT-аутентификацию с механизмом refresh-токенов и шифрованием паролей по алгоритму bcrypt.

Актуальность разработки обусловлена растущим спросом на цифровые решения в сфере здоровья и фитнеса, когда увеличивается значимость персонального контроля за физическим состоянием. Современные пользователи сталкиваются с проблемой информационной перегрузки и отсутствием персонализированных инструментов, способных адаптироваться к их индивидуальным особенностям и динамике прогресса. Существующие фитнес-приложения зачастую предлагают стандартизированные решения без учёта антропометрических данных, пищевых привычек и реальных возможностей пользователя, что снижает их эффективность и приводит к высокому проценту оттока.

Разрабатываемая платформа решает эти проблемы за счёт внедрения интеллектуальных алгоритмов генерации контента, интеграции AI-технологий для анализа визуальной информации и реализации системы динамической адаптации. Проект демонстрирует практические подходы к проектированию высоконагруженных серверных систем, обрабатывающих гетерогенные данные и обеспечивающих персонализированный пользовательский опыт в реальном времени, что соответствует современным требованиям к цифровым сервисам в области здравоохранения и wellness-индустрии.

1.2. Общее описание сервисов

На рынке представлены различные сервисы для контроля фитнеса и питания, такие как MyFitnessPal, Strong, 8fit, FatSecret и Яндекс.Здоровье. Эти платформы предлагают базовые функции учёта тренировок или питания, но их возможности сильно различаются по уровню автоматизации и персонализации.

Большинство сервисов фокусируются на одном аспекте: MyFitnessPal и FatSecret — на питании, Strong — на силовых тренировках, 8fit — на готовых программах. При этом они практически не предлагают интеллектуальных функций, таких как анализ питания по фото или динамическая адаптация программ.

Автоматическая генерация тренировок встречается редко и обычно представляет собой статические, неизменяемые программы. Сервисы не учитывают индивидуальный прогресс пользователя, доступное оборудование или предпочтения по месту занятий, что снижает их практическую ценность.

Технологии искусственного интеллекта для анализа визуального контента стали активно применяться в существующих решениях, но нет приложения, который вмещает в себя такое комплексное приложение, которое также создает тренировочные сессии на основе статистики пользователя.

Мотивационные системы и персонализированные рекомендации реализованы фрагментарно, без учёта комплексных данных о пользователе. Отсутствует единая экосистема, которая бы сопровождала пользователя от постановки цели до отслеживания прогресса с адаптацией рекомендаций.

Таким образом, современные фитнес-платформы предлагают разрозненные инструменты без глубокой интеграции и автоматизации. Это создаёт возможность для разработки комплексного решения, объединяющего интеллектуальный анализ данных, персонализированную генерацию контента и адаптивное сопровождение пользователя.

1.3. MyFitnessPal

Сервис MyFitnessPal является одним из наиболее популярных международных приложений для отслеживания питания и физической активности. Платформа ориентирована на пользователей, стремящихся контролировать вес, вести здоровый образ жизни и достигать фитнес-целей. MyFitnessPal поддерживает регистрацию через email и социальные сети, предоставляет стандартный функционал управления учётной записью, включая восстановление пароля и настройку профиля.

Основной сценарий использования MyFitnessPal заключается в ведении дневника питания с обширной базой продуктов, расчёте суточной калорийности и баланса нутриентов. Пользователи могут вручную добавлять приёмы пищи, сканировать штрих-коды продуктов, создавать собственные блюда и рецепты. Платформа также предлагает базовое отслеживание физической активности с возможностью синхронизации с фитнес-трекерами и умными часами.

В MyFitnessPal реализованы механизмы расчёта суточных норм калорий (BMR и TDEE) на основе антропометрических данных пользователя и его целей (похудение, поддержание веса, набор массы). Сервис предоставляет статистику прогресса, графики изменения веса и отчёты о потреблении питательных веществ.

В MyFitnessPal отсутствует ряд функций, которые рассматриваются в рамках разрабатываемой платформы:

1. **Автоматическая генерация тренировочных программ.**
   Платформа не создаёт персонализированные тренировочные планы. Пользователи могут лишь вручную добавлять упражнения или импортировать данные из других фитнес-приложений, но автоматическое составление программ на основе целей, уровня подготовки и доступного оборудования не реализовано.
2. **Анализ питания по фотографиям с использованием AI.**
   MyFitnessPal полагается на ручной ввод данных, сканирование штрих-кодов и выбор из базы продуктов. Отсутствует функциональность автоматического распознавания блюд по фотографии и определения их пищевой ценности с помощью компьютерного зрения.
3. **Динамическая адаптация уровня подготовки.**
   Сервис не отслеживает прогресс пользователя в тренировках и не корректирует сложность программ на основе фактической активности. Все изменения в планах пользователь должен вносить вручную.
4. **Учёт места тренировок и доступного оборудования.**
   При добавлении упражнений не учитывается, где пользователь тренируется (дома, в зале, на

улице) и какое оборудование ему доступно, что снижает практическую применимость рекомендаций.

5. **Система персонализированных рекомендаций на основе AI.**
   MyFitnessPal предоставляет лишь базовые советы по питанию и общие рекомендации, но не генерирует индивидуальные рекомендации по тренировкам, восстановлению и питанию на основе анализа поведения пользователя.
6. **Механизм прогрессивной нагрузки.**
   Отсутствует автоматическое увеличение сложности тренировок по мере прогресса пользователя. Принцип прогрессивной нагрузки не реализован на уровне алгоритмов платформы.

Таким образом, MyFitnessPal является зрелым и функциональным сервисом для контроля питания, но его возможности в части автоматизации тренировочного процесса, интеллектуального анализа данных и персонализации ограничены. Это делает его менее эффективным для пользователей, нуждающихся в комплексном решении, объединяющем управление питанием и тренировками с адаптацией к индивидуальному прогрессу.

1.4. FatSecret

Сервис FatSecret является специализированным приложением для учёта калорий и контроля веса, ориентированным преимущественно на аспект питания. Платформа предоставляет пользователям инструменты для ведения дневника питания с обширной базой продуктов, расчёта суточной калорийности и отслеживания потребления макронутриентов. FatSecret поддерживает регистрацию через email и интеграцию с фитнес-трекерами для учёта физической активности.

Основной сценарий использования FatSecret сосредоточен вокруг контроля питания: пользователи могут добавлять приёмы пищи через ручной ввод, использовать сканер штрих-кодов, создавать собственные рецепты и блюда. Платформа предлагает детальную статистику по потреблению белков, жиров, углеводов, а также микронутриентов. Сервис включает в себя функцию расчёта базового метаболизма (BMR) и суточной потребности в калориях на основе антропометрических данных пользователя.

В FatSecret реализованы базовые социальные функции: возможность делиться прогрессом с друзьями, участие в сообществе пользователей, публикация рецептов. Платформа также предоставляет инструменты для установки целей по весу, ведения графика изменений массы тела и формирования отчётов о питании за выбранный период.

В FatSecret отсутствует ряд функций, которые рассматриваются в рамках разрабатываемой платформы:

1. **Автоматическая генерация тренировочных программ.**
   Функционал тренировок ограничен базовыми, стандартизированными планами без возможности персонализации. Платформа не создаёт индивидуальные программы на основе целей пользователя, уровня подготовки или доступного оборудования. Генерация адаптивных тренировочных программ, учитывающих прогресс и меняющих сложность, полностью отсутствует.
2. **Анализ питания по фотографиям с использованием AI.**
   FatSecret полагается исключительно на ручной ввод данных, сканирование штрих-кодов и выбор из обширной базы продуктов. Отсутствует функциональность автоматического распознавания блюд по фотографии и определения их пищевой ценности с помощью технологий компьютерного зрения и искусственного интеллекта.
3. **Динамическая адаптация уровня подготовки.**
   Сервис не отслеживает прогресс пользователя в физической активности и не корректирует сложность тренировочных программ на основе фактических результатов. Все изменения в планах питания и активности требуют ручного вмешательства пользователя.
4. **Система персонализированных рекомендаций на основе AI.**
   FatSecret предоставляет лишь общие рекомендации по питанию и стандартные советы, но не генерирует индивидуальные рекомендации по тренировкам, восстановлению и оптимизации питания на основе анализа поведения, прогресса и предпочтений конкретного пользователя.
5. **Механизм прогрессивной нагрузки.**
   Отсутствует автоматическое увеличение сложности тренировок по мере адаптации организма пользователя. Принцип прогрессивной нагрузки не реализован на уровне алгоритмов платформы даже для имеющихся базовых тренировочных планов.
6. **Интеллектуальный учёт места тренировок и оборудования.**
   При наличии тренировочных модулей не учитывается, где пользователь планирует заниматься (дома, в зале, на улице) и какое оборудование ему доступно, что снижает практическую применимость предлагаемых упражнений.

Таким образом, FatSecret является эффективным инструментом для контроля питания и отслеживания калорий, но его возможности в области автоматизации тренировочного процесса, интеллектуального анализа данных и комплексной персонализации остаются крайне ограниченными.

1.5. Strong

Приложение Strong является специализированным инструментом для отслеживания силовых тренировок, популярным среди пользователей, занимающихся в тренажёрном зале. Сервис фокусируется на детальном учёте параметров выполнения упражнений: подходов, повторений, используемого веса, времени отдыха. Strong предоставляет обширную библиотеку упражнений с демонстрационными GIF-анимациями и возможностью создания пользовательских упражнений.

Основной сценарий использования Strong заключается в планировании и отслеживании силовых тренировок. Пользователи могут создавать собственные программы, добавлять упражнения, устанавливать целевые параметры для каждого подхода. Платформа визуализирует прогресс через графики увеличения рабочих весов, отслеживает личные рекорды и предоставляет таймер отдыха между подходами. Strong также поддерживает интеграцию с Apple Health для синхронизации данных о физической активности.

В Strong реализованы функции анализа прогресса: сравнение текущих результатов с предыдущими тренировками, отслеживание тенденций в увеличении нагрузок, формирование отчётов о выполненных объёмах работы. Интерфейс оптимизирован для быстрого внесения данных во время тренировки.

В Strong отсутствует ряд функций, которые рассматриваются в рамках разрабатываемой платформы:

1. **Автоматическая генерация тренировочных программ.**
   Пользователи должны самостоятельно создавать все тренировки, выбирая упражнения из каталога или добавляя свои. Алгоритмическая генерация персонализированных программ на основе целей (похудение, набор массы, поддержание формы), уровня подготовки, истории тренировок и доступного оборудования отсутствует. Платформа предоставляет лишь инструменты для ручного составления планов.
2. **Учёт питания и анализ пищевой ценности.**
   Strong полностью игнорирует аспект питания — в приложении нет функционала для ведения дневника питания, расчёта калорий, отслеживания нутриентов или управления макроэлементами. Это существенное ограничение, поскольку питание составляет критически важную часть достижения фитнес-целей.
3. **Анализ питания по фотографиям с использованием AI.**
   Отсутствует полностью, так как приложение не работает с пищевыми данными.

Пользователи не могут загружать фотографии еды для автоматического определения её пищевой ценности.

4. **Динамическая адаптация уровня подготовки.**
   Хотя Strong отслеживает прогресс в весах и повторениях, автоматическая корректировка программ на основе этого прогресса не реализована. Пользователь должен самостоятельно анализировать статистику и вручную вносить изменения в тренировочный план для обеспечения прогрессивной нагрузки.
5. **Персонализированные рекомендации на основе AI.**
   Отсутствуют какие-либо системы интеллектуальных рекомендаций по тренировкам, восстановлению, периодизации нагрузок или профилактике травм. Все решения о планировании тренировочного процесса пользователь принимает самостоятельно.
6. **Учёт места тренировок и доступного оборудования.**
   Хотя Strong содержит упражнения для разных условий, он не адаптирует программы на основе доступного пользователю оборудования или предпочтительного места тренировок (дом, зал, улица).
7. **Система мотивации и напоминаний.**
   Функционал уведомлений ограничен базовыми напоминаниями о запланированных тренировках, без персонализированной мотивационной составляющей или контекстных советов.

Таким образом, Strong является эффективным инструментом для опытных пользователей, которые самостоятельно планируют свои тренировки и нуждаются в детальном отслеживании силовых показателей, но его возможности в области автоматизации, интеграции питания и интеллектуальной поддержки принятия решений остаются минимальными.

1.6. Яндекс.Здоровье

Российский сервис Яндекс.Здоровье представляет собой комплексную платформу для отслеживания различных аспектов здоровья, интегрированную в экосистему Яндекс. Сервис объединяет модули для контроля питания, физической активности, медицинских показателей и wellness-привычек.

Основной сценарий использования Яндекс.Здоровье заключается в комплексном мониторинге состояния здоровья: пользователи могут вести дневник питания, отслеживать шаги и активность через интеграцию с Яндекс.Картами и фитнес-трекерами, записывать медицинские показатели (давление, уровень глюкозы, вес), а также получать общие рекомендации по здоровому

образу жизни. Платформа поддерживает синхронизацию с некоторыми медицинскими устройствами и сторонними приложениями.

В Яндекс.Здоровье реализованы функции формирования отчётов о динамике ключевых показателей здоровья, установки целей по активности и питанию, получения персонализированных статей и советов на основе введённых данных. Сервис также предоставляет возможность консультаций с врачами через партнёрские медицинские сервисы.

В Яндекс.Здоровье отсутствует ряд функций, которые рассматриваются в рамках разрабатываемой платформы:

1. **Автоматическая генерация тренировочных программ.**
   Яндекс.Здоровье фокусируется на отслеживании общей физической активности (шаги, кардио), но полностью отсутствует функционал создания, планирования или генерации структурированных тренировочных программ, особенно силовых или специализированных. Пользователь не может получить персонализированный план упражнений.
2. **Анализ питания по фотографиям с использованием AI.**
   Ведение дневника питания осуществляется исключительно через ручной ввод продуктов или выбор из базы. Распознавание блюд по фотографиям с помощью компьютерного зрения не поддерживается, что существенно усложняет процесс точного учёта потребляемой пищи.
3. **Система персонализированных рекомендаций на основе AI для тренировок.**
   Рекомендации носят общий характер (статьи о здоровье, советы по питанию) и не генерируются индивидуально для каждого пользователя на основе анализа его тренировочного прогресса, целей или особенностей организма. Отсутствуют интеллектуальные рекомендации по планированию тренировок, периодизации или восстановлению.
4. **Учёт места тренировок и доступного оборудования.**
   Не реализовано, так как сервис не занимается составлением тренировочных программ и не предлагает упражнений, адаптированных под разные условия тренировок.
5. **Механизм прогрессивной нагрузки.**
   Отсутствует ввиду отсутствия функционала управления тренировками. Сервис не отслеживает прогресс в силовых показателях или выносливости и не может корректировать нагрузки соответственно.

Таким образом, Яндекс.Здоровье позиционируется как общая платформа для здоровья и wellness, но не предоставляет специализированных инструментов для серьёзных фитнес-

занятий, персонализированного тренировочного планирования или интеллектуального анализа питания, что оставляет нишу для специализированных фитнес-решений.

1.7. 8fit

Приложение 8fit предлагает готовые тренировочные программы и планы питания, ориентированные в основном на домашние тренировки без специального оборудования. Сервис сочетает в себе элементы фитнес-тренировок и диетических рекомендаций, предоставляя пользователям структурированный подход к здоровому образу жизни.

Основной сценарий использования 8fit заключается в выполнении ежедневных тренировок из предлагаемых программ и следовании рекомендованным планам питания. Платформа предоставляет таймеры для интервальных тренировок, возможность адаптации интенсивности занятий под текущий уровень подготовки. 8fit также включает функцию составления списка покупок на основе выбранного плана питания.

В 8fit реализована система целей (похудение, поддержание тонуса, увеличение энергии) и соответствующая под них подборка тренировок и рецептов. Платформа предлагает различные программы продолжительностью от нескольких недель до месяцев, с постепенным увеличением сложности по заранее определённому расписанию.

В 8fit отсутствует ряд функций, которые рассматриваются в рамках разрабатываемой платформы:

1. **Динамическая адаптация уровня подготовки на основе прогресса.**
   Программы 8fit являются статичными и не адаптируются к фактическому прогрессу пользователя автоматически. Увеличение сложности происходит по фиксированному расписанию, а не на основе анализа выполнения упражнений, скорости восстановления или достигнутых результатов. Платформа не оценивает, насколько успешно пользователь справляется с текущими нагрузками.
2. **Анализ питания по фотографиям с использованием AI.**
   Питание контролируется через готовые планы и рецепты, предоставляемые приложением. Автоматическое распознавание реальных блюд, которые потребляет пользователь, по фотографиям отсутствует. Пользователь ограничен предложенными рецептами и не может легко добавлять собственные приёмы пищи с автоматическим определением их пищевой ценности.

3. **Персонализированные рекомендации на основе AI.**
   Рекомендации в 8fit стандартизированы и привязаны к выбранной программе. Они не генерируются индивидуально на основе анализа текущих данных пользователя: его прогресса в тренировках, фактического питания, антропометрических изменений, уровня энергии или субъективных ощущений.
4. **Учёт различных мест тренировок и доступного оборудования.**
   8fit ориентирован преимущественно на домашние тренировки с минимальным оборудованием или без него. Платформа не учитывает возможность тренировок в тренажёрном зале, на улице или со специализированным оборудованием, что ограничивает её применение для пользователей с разными предпочтениями и доступными ресурсами.
5. **Механизм прогрессивной нагрузки, основанный на индивидуальных показателях.**
   Увеличение сложности происходит по универсальному для всех пользователей расписанию, а не на основе индивидуальных показателей силы, выносливости или техники выполнения упражнений. Это может приводить к недостаточной или избыточной нагрузке для конкретного пользователя.
6. **Автоматическая генерация полностью персонализированных тренировок.**
   Хотя 8fit предлагает различные программы, они не генерируются индивидуально под конкретного пользователя с учётом его уникальных параметров, истории травм, предпочтений в типах упражнений или неравномерностей в развитии разных мышечных групп.

Таким образом, 8fit предоставляет структурированный, но стандартизированный подход к тренировкам и питанию, не обладая гибкостью и адаптивностью, необходимыми для истинно персонализированного фитнес-сопровождения.

1.8. Сравнительный анализ

При обзоре аналогов были выявлены следующие значимые критерии для сравнительного анализа:

1. **Управление тренировками**

Данный критерий отражает базовую функциональность работы с тренировочными программами, включая создание, редактирование, просмотр и отслеживание истории выполнения тренировок. Также есть возможность пропустить упражнение, что отобразится в истории тренировки. Управление тренировками охватывает такие аспекты как назначение упражнений на желательные группы мышц, управлением интенсивности тренировки и также

место проведения тренировки. Этот критерий является фундаментальным для любого фитнес-приложения, так как обеспечивает основу для систематического подхода к физическим нагрузкам. Качество реализации оценивается в гибкости настройки параметров и полноте предоставляемой статистики.

2. Автоматическая генерация тренировок

Данный критерий отражает способность системы создавать персонализированные тренировочные программы без необходимости ручного составления со стороны пользователя или тренера. Наличие этой функции критически важно для пользователей, не обладающих достаточными знаниями в области фитнеса, а также для тех, кто стремится минимизировать временные затраты на планирование тренировочного процесса. Автоматическая генерация должна учитывать индивидуальные параметры пользователя (уровень подготовки, цели, место проведения) и динамически адаптироваться на основе прогресса. Отсутствие этой функции в аналогах существенно снижает ценность платформы для начинающих и занятых пользователей.

3. Учёт питания

Критерий обозначает наличие функционала для отслеживания потребляемых продуктов, включая ведение дневника питания, расчёт суточной калорийности и баланса нутриентов (белков, жиров, углеводов). Учёт питания является фундаментальным компонентом любой системы управления здоровьем, поскольку питание составляет большую часть успеха в достижении фитнес-целей. Качество реализации этого критерия оценивается по удобству ввода данных, полноте базы продуктов и возможностям анализа.

4. Анализ питания по фото

Данный критерий отражает использование технологий компьютерного зрения и искусственного интеллекта для автоматического определения пищевой ценности блюда по фотографии. Эта функция значительно упрощает процесс учёта питания, устраняя необходимость ручного поиска продуктов в базе данных или взвешивания. Наличие анализа по фото является ключевым конкурентным преимуществом, демонстрирующим технологическую продвинутость платформы и ориентированность на удобство пользователя.

5. Динамическая адаптация уровня

Критерий обозначает способность системы автоматически корректировать уровень сложности тренировок и рекомендаций на основе фактической активности и прогресса пользователя. В отличие от статических программ, динамическая адаптация обеспечивает

непрерывное соответствие нагрузок текущим возможностям пользователя, предотвращая плато в результатах и снижая риск травм. Этот критерий важен для обеспечения долгосрочной эффективности тренировочного процесса.

6. Учёт места тренировок

Под данным критерием понимается возможность учёта доступного пользователю пространства и оборудования (домашние условия, тренажёрный зал, улица) при генерации тренировочных программ. Учёт места тренировок делает рекомендации практичными и реализуемыми, повышая приверженность пользователя к программе. Отсутствие этого критерия может привести к формированию невыполнимых программ, требующих недоступного оборудования.

7. Система напоминаний/мотивации

Критерий отражает наличие механизмов для поддержания вовлечённости пользователя через push-уведомления, email-рассылки, SMS или внутриприкладные напоминания. Эффективная система мотивации способствует формированию привычки, повышает частоту использования приложения и снижает процент оттока пользователей. Качество реализации оценивается по персонализации, контекстной релевантности и поддержке различных каналов коммуникации.

8. Персонализированные рекомендации

Данный критерий обозначает генерацию индивидуальных советов по питанию, тренировкам и восстановлению на основе анализа данных конкретного пользователя (его целей, прогресса, пищевых привычек). Персонализация отличает интеллектуальную платформу от шаблонных решений и повышает ценность для пользователя. Рекомендации могут варьироваться от простых напоминаний до сложных аналитических выводов, сгенерированных с помощью AI.

9. Прогрессивная нагрузка

Критерий отражает реализацию принципа постепенного увеличения интенсивности, объёма или сложности тренировок для обеспечения постоянного прогресса и адаптации организма. Прогрессивная нагрузка является научно обоснованным методом в спортивной физиологии, необходимым для достижения результатов в силовом тренинге, выносливости и других направлениях. Наличие этого критерия свидетельствует о качественной методологической основе тренировочных алгоритмов платформы.

Все выбранные критерии были сведены в итоговую сравнительную таблицу 1.

Таблица 1. Сравнение аналогов сервиса.

	My App	MyFitness Pal[10]	FatSecret[11]	Strong[12]	Яндекс.Здоровье[13]	8fit[14]
Управление тренировками	да	да	нет	да	да	да
Автоматическая генерация тренировок	да	нет	нет	нет	нет	да
Учет питания	да	да	да	нет	да	да
Анализ питания по фото	да	нет	нет	нет	нет	нет
Динамическая адаптация уровня	да	нет	нет	нет	нет	нет
Учет места тренировок	да	нет	нет	да	нет	да
Система напоминаний/мотивации	да	да	да	да	да	да
Персонализированные рекомендации	да	да	нет	нет	да	нет
Прогрессивная нагрузка	да	нет	нет	да	нет	да
Итого количество «да» в столбце	**9**	**4**	**2**	**4**	**4**	**6**

Выводы по главе

По результатам проведённого анализа можно сделать вывод, что на рынке фитнес-приложений и сервисов контроля здоровья представлено значительное количество решений, таких как MyFitnessPal, Strong, 8fit, FatSecret и Яндекс.Здоровье. Эти платформы предлагают базовые инструменты для учёта тренировок или питания, но их функциональность остаётся узкоспециализированной и фрагментарной.

Наиболее развитые сервисы, такие как MyFitnessPal, сосредоточены на глубоком анализе питания, в то время как другие (Strong, 8fit) специализируются на тренировочных программах. Однако ни один из рассмотренных аналогов не предоставляет комплексного решения, объединяющего оба этих критически важных аспекта в единую адаптивную систему.

Существенным недостатком всех анализируемых платформ является отсутствие интеллектуальных функций автоматизации. В них не реализованы автоматическое создание персонализированных тренировок, анализ питания по фотографиям с использованием AI, динамическая адаптация программ на основе прогресса пользователя или система интеллектуальных рекомендаций. Технологии искусственного интеллекта с рекомендациями, анализа питания по фотографиям применяются фрагментарно либо отсутствуют полностью, что не позволяет говорить о комплексном подходе к автоматизации фитнес-сопровождения.

Таким образом, в рамках анализа было рассмотрено 5 ведущих фитнес-сервисов и выделено 9 ключевых критериев для сравнительного анализа. Результаты сравнения показывают, что существующие решения покрывают лишь отдельные аспекты контроля здоровья, но не предлагают сквозной автоматизации и глубокой персонализации пользовательского опыта. Это подтверждает обоснованность разработки инновационной платформы, ориентированной на интеграцию AI-технологий, адаптивную генерацию контента и комплексное сопровождение пользователя.

На основе проведенного сравнительного анализа было однозначно определено, что **разработка является актуальной**.

Глава 2. Проектирование системы и технологии разработки

2.1. Основной функционал сервисов

Сервис Auth должен реализовывать следующие функции

a) Регистрация нового пользователя с указанием корректных персональных данных, включающих в себя номер телефона и почту.
b) Аутентификация пользователя по логину и паролю с выдачей JWT-токена доступа и refresh-токена
c) Обновление пары access/refresh токенов через отдельный endpoint
d) Верификация email и телефона пользователя
e) Восстановление доступа к аккаунту через привязанный email или телефон

Сервис Executor должен реализовывать следующие функции

a) Получение и обработка отложенных задач на отправку уведомлений
b) Управление очередью отложенных задач
c) Интеграция с внешними провайдерами SMS и email для доставки сообщений
d) Отправка push-уведомлений через интеграцию с мобильной платформой
e) Логирование статуса выполнения всех отправленных уведомлений

Сервис Recommendation должен реализовывать следующие функции

a) Фоновая генерация персонализированных ежедневных рекомендаций на основе цели пользователя и его текущего прогресса
b) Контекстно-зависимые советы
c) Ручная генерация рекомендаций по запросу пользователя через отдельный endpoint
d) Интеграция с Open AI для расширенной генерации текстовых советов
e) Управление историей выданных рекомендаций с возможностью отметки о прочтении

Сервис Nutrition должен реализовывать следующие функции

a) Загрузка фотографий еды в S3-бакет и их анализ через Open AI для определения пищевой ценности (КБЖУ)
b) Периодическая фоновая генерация питания
c) Ручной ввод приёмов пищи с указанием названия блюда, количества и пищевой ценности
d) Ведение дневника питания с группировкой по приёмам пищи и датам

e) Расчёт суммарного суточного потребления калорий и нутриентов с сравнением с целевыми показателями
f) Создавать новую отложенную задачу на уведомление о генерации питания
g) Формирование отчётов по питанию за выбранный период

Сервис CRUD должен реализовывать следующие функции

a) Управление справочником упражнений
b) Управление пользовательской информацией
c) Управление параметрами пользователя
d) Управление историей веса
e) Управление калориями пользователя
f) Управление пищевыми записями
g) Управление отложенными заданиями
h) Управление упражнениями в тренировках

Сервис Workouts должен реализовывать следующие функции

a) Автоматическая генерация персонализированных тренировок на основе информации о пользователе
b) Периодическая фоновая генерация тренировок с проверкой наличия активных незавершённых тренировок и соблюдения времени отдыха
c) Учёт предпочтений пользователя (популярные типы упражнений и места тренировок) на основе анализа истории
d) Применение принципа прогрессивной нагрузки с увеличением сложности при успешном выполнении серии тренировок
e) Отметка о выполнении тренировки с возможностью указания фактически потраченных калорий и времени
f) Создавать новую отложенную задачу на уведомление о генерации тренировки

g) Просмотр и фильтрация истории тренировок по статусу, дате и уровню сложности

h) Расчёт прогнозируемых энергозатрат для каждой сгенерированной тренировки

Сервис Avatar должен реализовывать следующие функции

a) Генерация ссылки на загрузку фотографии внутрь S3-бакета для загрузки пользовательской фотографии.
b) Возможность обновление/удаления фотографии из профиля пользователя.

Более подробно функциональные требования описаны в п. 4.1. Технического задания.

2.2. Пользовательские сценарии

Для формализации функциональных требований и наглядного представления взаимодействия пользователей с системой в проекте разработаны диаграммы прецедентов UML. Эти диаграммы визуализируют все функциональные возможности системы, описанные в техническом задании, и демонстрируют, как каждый модуль реализует соответствующие требования.

2.2.1. Ключевые пользовательские сценарии

Сценарий 1: Первичная настройка системы

Пользователь начинает работу с приложением через процесс регистрации (Рис. 6), в ходе которого создаёт учётную запись с email и паролем. После верификации email пользователь заполняет детальную анкету профиля через модуль CRUD (Рис. 4), указывая антропометрические данные (рост, текущий вес), фитнес-цель (lose_weight, build_muscle, stay_fit) и уровень повседневной активности (not_active, active, sportive). На основе этих данных система автоматически рассчитывает суточную норму калорий (TDEE) и формирует базовые настройки.

Сценарий 2: Ежедневный контроль питания

Ежедневно пользователь использует модуль питания (Рис. 1) для учёта рациона. Основной механизм – загрузка фотографий приёмов пищи, которые автоматически анализируются через интеграцию с Open AI API для определения пищевой ценности (калории, белки, жиры, углеводы). Альтернативно пользователь может вручную вводить данные о потреблённой пище. Система агрегирует все записи за день, рассчитывает суммарный баланс КБЖУ и сравнивает его с целевыми показателями, предоставляя наглядную сводку

Сценарий 3: Тренировочный процесс

На основе данных профиля модуль тренировок (Рис. 3) в фоновом режиме генерирует персонализированные тренировочные программы. Алгоритм учитывает цель пользователя, текущий уровень подготовки, доступное оборудование и предпочтительное место занятий (дом, тренажёрный зал, улица). Пользователь получает push-уведомление (через модуль Executor, Рис. 5) о готовности новой тренировки, выполняет её, отмечая прогресс в реальном времени, и завершает сессию. Система применяет принцип прогрессивной нагрузки, анализируя успешность выполнения для корректировки сложности будущих тренировок.

Сценарий 4: Получение интеллектуальных рекомендаций

Модуль рекомендаций (Рис. 2) ежедневно анализирует данные пользователя: потребление питания из модуля Nutrition, выполнение тренировок из модуля Workouts, динамику веса из модуля CRUD. На основе этого анализа генерируются персонализированные рекомендации, которые могут касаться корректировки рациона, оптимизации тренировочного процесса, советов по восстановлению или мотивационных сообщений. Для сложных формулировок используется интеграция с OpenAI.

Сценарий 5: Мониторинг прогресса и корректировка целей

Через модуль CRUD операций (Рис. 4) пользователь имеет доступ ко всем своим данным: может просматривать историю веса с построением графиков динамики, анализировать выполненные тренировки за различные периоды, изучать пищевые отчёты. На основе этой аналитики пользователь может корректировать свои цели и параметры, что немедленно отражается на генерации новых тренировок и рекомендаций.

Сценарий 6: Управление профилем и безопасность

Модуль авторизации (Рис. 6) обеспечивает безопасный доступ ко всем функциям системы через JWT-токены с механизмом обновления. Модуль аватарки (Рис. 7) позволяет пользователю загружать и обновлять фотографию профиля через безопасную прямую загрузку в S3-хранилище. В случае утери доступа система предоставляет процедуру восстановления через верифицированные email или телефон.

2.2.2. Диаграммы прецендентов

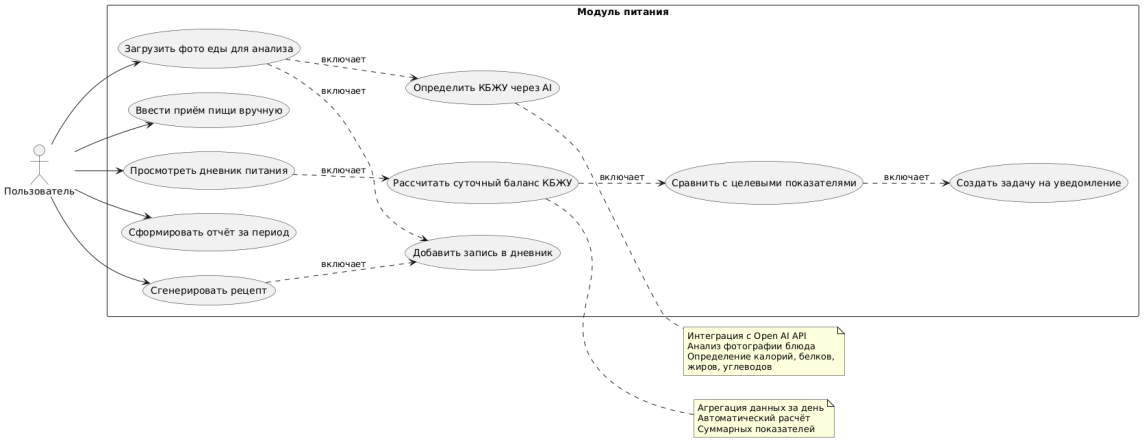

Рис. 1. Диаграмма прецендентов модуля питания

Рис. 2. Диаграмма прецендентов модуля рекомендаций

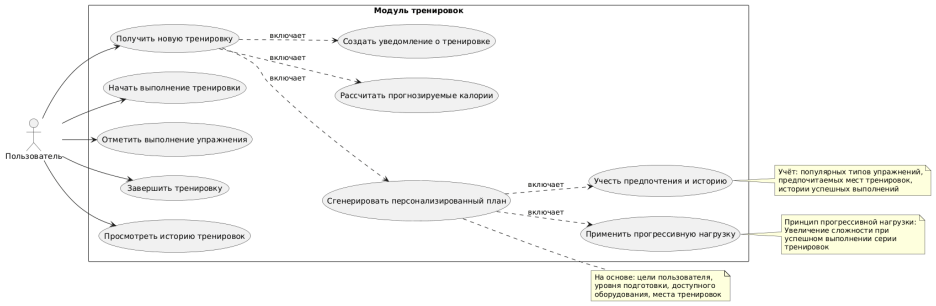

Рис. 3. Диаграмма прецендентов модуля тренировок

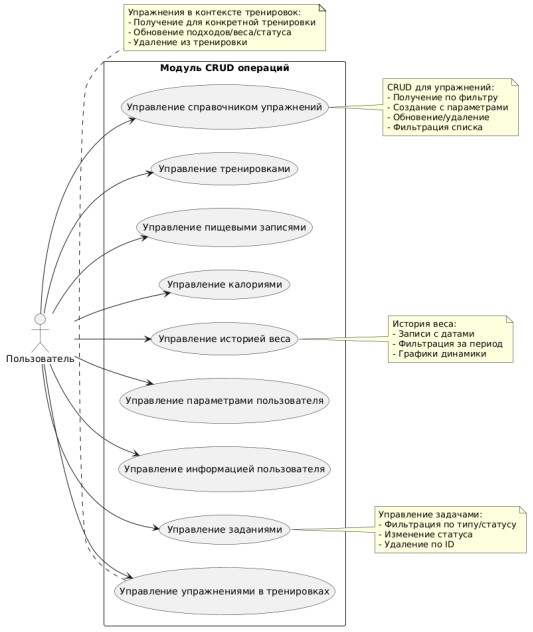

Рис. 4. Диаграмма прецендентов модуля CRUD операций

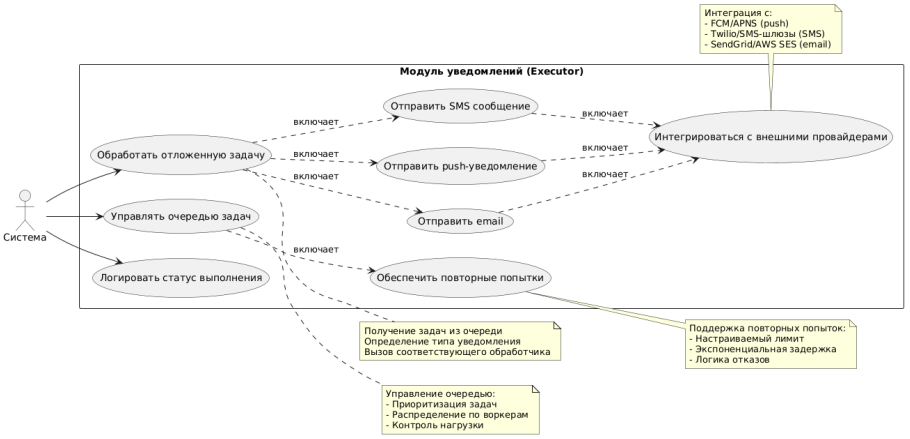

Рис. 5. Диаграмма прецендентов модуля отложенных задач

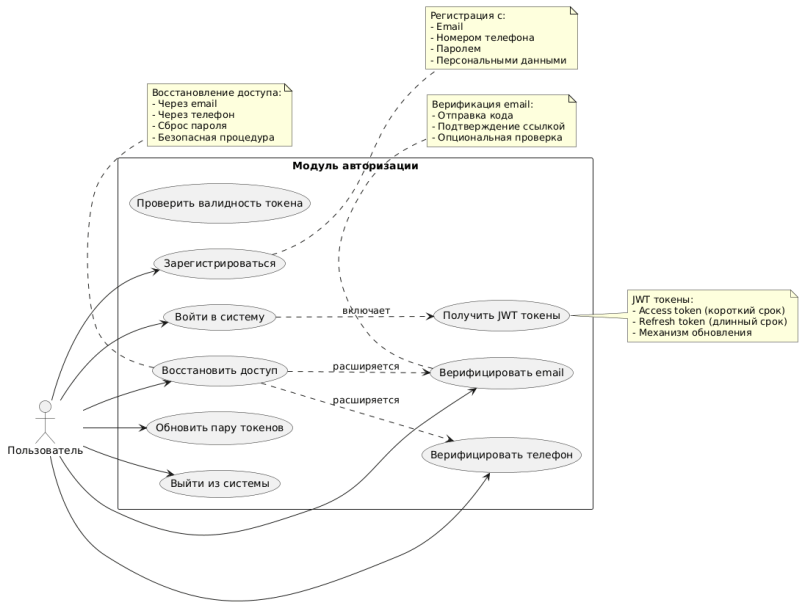

Рис. 6. Диаграмма прецендентов модуля авторизации

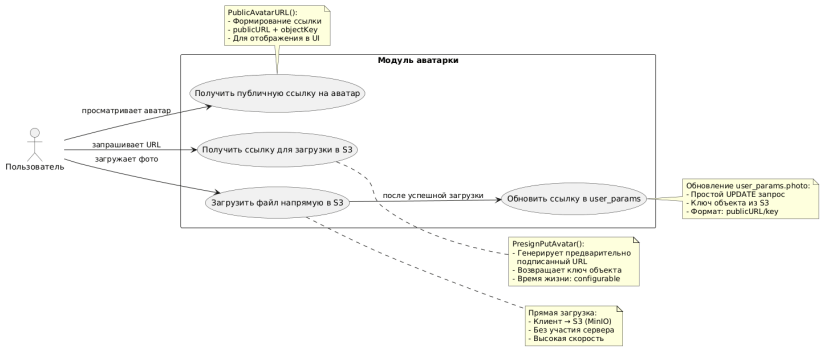

Рис. 7. Диаграмма прецендентов модуля аватарки

2.2.3. Валидация полноты реализации

Каждый прецедент на диаграммах соответствует конкретной функциональной возможности системы, что обеспечивает:

1. Непротиворечивость: Отсутствие конфликтов между различными функциями
2. Интегрированность: Все модули связаны через общие данные и процессы
3. Практическую реализуемость: Каждый сценарий технически осуществим в рамках выбранной архитектуры

Исходя из вышеперечисленных пользовательских сценариев и их визуализации через диаграммы прецедентов были составлены детальные функциональные требования, с которыми подробно можно ознакомиться в п. 4.1 Технического задания.

2.3. Архитектура решения

2.3.1. Обоснование выбора архитектуры

Платформа персонализированного контроля здоровья и фитнеса объединяет несколько сложных, но взаимосвязанных доменов: управление пользователями и аутентификацию, генерацию адаптивных тренировок, анализ питания с использованием AI, формирование рекомендаций и систему уведомлений. Для эффективной реализации данного проекта в рамках курсовой работы была выбран**а** модульная монолитная архитектура с чётким разделением на слои по принципам Clean Architecture и выделенными доменными контекстами в соответствии с Domain-Driven Design (DDD)**.**

2.3.2. Ключевые аргументы в пользу монолитной архитектуры для данного проекта:

1. **Оптимальное соответствие масштабу проекта:** Разрабатываемое приложение представляет собой MVP, создаваемое силами одного разработчика в рамках учебного проекта. Для такого масштаба микросервисная архитектура создаёт избыточную сложность без объективной необходимости, увеличивая время разработки и требуя дополнительных ресурсов на поддержку распределённой инфраструктуры.
2. **Отсутствие сетевых издержек:** Все компоненты системы работают в едином процессе, что полностью исключает сетевые задержки при межмодульном взаимодействии. Для приложения, требующего быстрой обработки пользовательских данных в реальном времени (генерация тренировок, мгновенный анализ питания), это является критически важным преимуществом. В микросервисной архитектуре каждый вызов между сервисами добавляет задержку, которая накапливается в сложных пользовательских сценариях.
3. **Упрощение разработки и сопровождения:** Единая кодовая база и согласованная модель данных значительно упрощают процесс разработки, отладки и сопровождения системы. Это позволяет сосредоточиться на реализации бизнес-логики, а не на решении проблем, характерных для распределённых систем: согласованности данных, версионности API, распределённых транзакций и отладки межсервисных взаимодействий.
4. **Экономия вычислительных ресурсов:** Монолитная архитектура минимизирует накладные расходы, связанные с контейнеризацией отдельных сервисов, оркестрацией контейнеров, реализацией сервисной сетки (service mesh) и организацией сложных систем мониторинга распределённой системы.

2.3.3. Архитектурная организация монолита:

Система построена как **модульный монолит**, где чёткое разделение на слои Clean Architecture обеспечивает высокую связность внутри доменных контекстов и низкую связанность между слоями:

- **Domain-слой** содержит бизнес-сущности и чистую бизнес-логику (User, Workout, Nutrition, Recommendation), полностью независимые от инфраструктурных деталей.
- **Application-слой** реализует use cases (сценарии использования) и координирует взаимодействие между доменными объектами.
- **Infrastructure-слой** обеспечивает взаимодействие с внешними системами: PostgreSQL для хранения данных, Redis для кэширования, S3-совместимое хранилище для медиафайлов и Open AI API для анализа изображений.
- **Delivery-слой** предоставляет REST API через фреймворк Gin, выступая точкой входа для клиентских приложений.

2.3.4. Преимущества комбинации DDD и Clean Architecture:

Применение принципов Domain-Driven Design в сочетании с Clean Architecture обеспечивает несколько ключевых преимуществ:

1. **Чёткие границы ответственности:** Каждый ограниченный контекст (Bounded Context) инкапсулирует свою специфическую бизнес-логику, что упрощает понимание, тестирование и развитие системы.
2. **Подготовленность к эволюции:** Не смотря на выбор монолитной архитектуры, система спроектирована таким образом, что каждый доменный контекст представляет собой логически завершённый модуль. В случае необходимости масштабирования или выделения отдельных компонентов в будущем, эти модули могут быть относительно легко преобразованы в самостоятельные микросервисы без полной переработки архитектуры.
3. **Локализация изменений:** Изменения в бизнес-логике одного домена минимально затрагивают другие части системы благодаря строгому разделению слоёв и чётким контрактам между ними.
4. **Тестируемость:** Изоляция доменной логики от инфраструктурных деталей позволяет эффективно тестировать бизнес-правила с помощью модульных тестов без зависимости от внешних систем.

2.3.5. Архитектурная перспектива:

Разрабатываемая система с архитектурной точки зрения представляет собой **самодостаточный модуль**, который в контексте более крупной экосистемы мог бы выступать в роли специализированного микросервиса. Однако для текущего этапа — создания полнофункционального MVP в рамках ВКР — монолитная архитектура является наиболее рациональным выбором, оптимально балансирующим между скоростью разработки, простотой сопровождения и архитектурной чистотой. Такой подход позволяет сосредоточить усилия на реализации уникальной ценности продукта — интеллектуальных алгоритмах персонализации тренировок и анализа питания — не отвлекаясь на преодоление сложностей, характерных для распределённых систем на ранних этапах развития проекта.

2.3.6. Технологический стек и обоснование выбора

1) Язык программирования Go

Выбор Go 1.21+ обусловлен следующими факторами:

- **Производительность:** Статическая компиляция в нативный код, эффективное управление памятью
- **Простота и читаемость:** Минималистичный синтаксис, явная обработка ошибок
- **Встроенная поддержка конкурентности:** Горутины и каналы для эффективной работы с фоновыми задачами
- **Богатая экосистема:** Библиотеки для работы с PostgreSQL (pgx), Redis (go-redis), HTTP-фреймворки (Gin)
- **Кроссплатформенность:** Единый код может быть скомпилирован для различных операционных систем
- **Поддержка в индустрии:** Активно развивающийся язык с сильным комьюнити

2) Кэширование на Redis

Redis 7+ используется для:

- **Кэширования результатов запросов:** Часто запрашиваемые данные (справочники упражнений, шаблоны тренировок)
- **Хранения сессий пользователей:** JWT-токены, временные данные сессии
- **Очередей задач:** Для модуля Executor
- **Pub/Sub:** Для событийной модели взаимодействия между модулями

3) Хранение файлов в S3/MinIO

Объектное хранилище используется для:

- **Фотографий еды:** Загружаемые пользователями изображения для анализа питания
- **Аватаров пользователей:** Фотографии профилей
- **Демонстрационных материалов:** GIF-анимации упражнений
- **Архивов отчётов:** Сформированные отчёты за периоды

Преимущества S3-совместимого хранилища:

- **Масштабируемость:** Практически неограниченный объём хранения
- **Доступность:** Высокая доступность и durability данных
- **Экономичность:** Оплата только за фактически используемое пространство
- **Интеграция:** Простая интеграция через стандартные API

4) Внешние интеграции

- **OpenAI API:** Для анализа фотографий еды (Computer Vision) и генерации текстовых рекомендаций
- **Внешние провайдеры уведомлений:** Для отправки SMS, email и push-уведомлений
- **Сторонние сервисы аутентификации:** (опционально) OAuth-провайдеры для социальных сетей

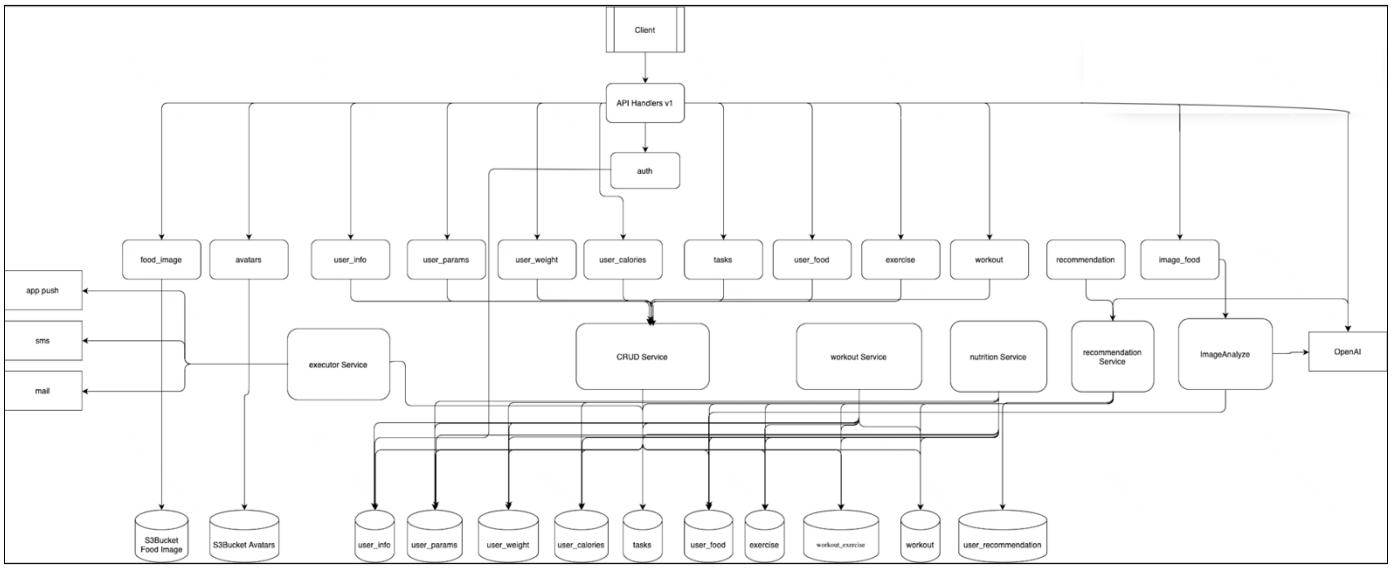

Рис.8. Диаграмма архитектуры системы

2.4. Проектирование сервиса

2.4.1. Общая архитектура системы

Система (Рис. 8) построена по принципам **Clean Architecture** и **Domain-Driven Design (DDD)** со следующими архитектурными слоями:

- **Domain-слой** — содержит чистую бизнес-логику и сущности (User, Workout, Exercise, Nutrition).
- **Application-слой** — реализует use cases и координирует взаимодействие между доменными объектами.
- **Infrastructure-слой** — обеспечивает взаимодействие с внешними системами (БД, кэш, внешние API).
- **Delivery-слой** — предоставляет REST API для клиентских приложений.

2.4.2. Ключевые сервисы и их взаимодействие

Система состоит из семи основных сервисов:

1. **Сервис аутентификации (Auth Service)** — управление пользователями, JWT-аутентификация, верификация.
2. **Сервис тренировок (Workout Service)** — генерация персонализированных тренировок, прогрессивная нагрузка.
3. **Сервис питания (Nutrition Service)** — анализ фотографий еды через OpenAI, учёт калорий.
4. **Сервис рекомендаций (Recommendation Service)** — генерация персонализированных советов на основе данных пользователя.
5. **Сервис отложенных задач (Executor Service)** — управление очередью уведомлений (email, SMS, push).
6. **CRUD-сервис** — базовые операции для всех сущностей системы.
7. **Сервис аватаров (Avatar Service)** — управление загрузкой изображений в S3-хранилище.

2.4.3. Модель данных и хранение

База данных спроектирована в PostgreSQL 15+ с использованием схемы bodyfuel. Основные компоненты:

- **Пользователи и профиль** (user_info, user_params, user_weight) — учётные данные, антропометрические данные, история веса.
- **Тренировки** (workout, exercise, workout_exercise) — программы тренировок, каталог упражнений.

- **Питание и мониторинг** (user_food, user_calories, user_recommendation) — дневник питания, калории, рекомендации.
- **Инфраструктура** (tasks, user_refresh_token) — очередь задач, управление сессиями.

Для повышения производительности используется:

- **Redis** для кэширования часто запрашиваемых данных и хранения сессий
- **S3-совместимое хранилище** для медиафайлов (фотографии еды, аватары, GIF упражнений)

2.4.4. Внешние интеграции

1. **OpenAI API** — для анализа фотографий еды (Computer Vision) и генерации текстовых рекомендаций.
2. **Внешние провайдеры уведомлений** — SendGrid/Mailgun для email, Twilio для SMS, Firebase для push-уведомлений.
3. **Сторонние сервисы аутентификации** (опционально) — OAuth-провайдеры (Google, Apple).

2.4.5. Безопасность и производительность

- **JWT-аутентификация** с короткоживущими access-токенами и долгоживущими refresh-токенами.
- **Хеширование паролей** с использованием bcrypt.
- **Валидация всех входящих данных** на уровне API и бизнес-логики.
- **Оптимизированные индексы** в БД для частых запросов (история тренировок, питание по датам).
- **Асинхронная обработка** длительных операций (анализ фото, генерация тренировок) через очередь задач.

2.4.6. Масштабируемость и отказоустойчивость

Архитектура системы предусматривает возможность горизонтального масштабирования:

- Репликация БД PostgreSQL для распределения нагрузки чтения.
- Кластеризация Redis для увеличения доступности кэша.
- Возможность выделения отдельных модулей в микросервисы при росте нагрузки.

Все критические операции (генерация тренировок, анализ питания) реализованы с механизмами повторных попыток и логированием ошибок для обеспечения отказоустойчивости.

2.5. Проектирование базы данных

2.5.1. Обоснование выбора PostgreSQL и стратегии проектирования

В качестве основной СУБД выбран **PostgreSQL 15+**, что обусловлено следующими техническими и функциональными требованиями системы:

1. **Транзакционная целостность ACID:** Система работает с критически важными данными о здоровье пользователей. Гарантии атомарности, консистентности, изоляции и долговечности транзакций PostgreSQL обеспечивают невозможность потери или повреждения данных при одновременных операциях обновления веса, завершения тренировок и записи питания.
2. **Поддержка доменных типов ENUM:** Наличие 7 перечислимых типов (want, lifestyle, workouts_status и др.) позволяет на уровне схемы БД гарантировать валидность данных. Например, значение поля wants может быть только lose_weight, build_muscle или stay_fit, что исключает ошибки приложения.
3. **Эффективная работа со временными рядами:** Требования к хранению истории веса, питания и тренировок предполагают частые операции вставки и агрегации временных данных. Оптимизированная работа с типами TIMESTAMPTZ и INTERVAL в PostgreSQL идеально подходит для таких сценариев.
4. **Производительность сложных JOIN-запросов:** Алгоритмы генерации тренировок требуют объединения данных из 5-7 таблиц (пользователь, параметры, история тренировок, упражнения). Оптимизатор запросов PostgreSQL эффективно строит планы выполнения для таких сложных запросов.
5. **Масштабируемость вертикальная и горизонтальная:** Возможность добавления реплик только для чтения позволяет распределять нагрузку аналитических запросов (отчёты, статистика) без воздействия на транзакционную нагрузку.

2.5.2. Общая структура схемы данных

Все таблицы размещены в схеме bodyfuel, что обеспечивает логическое отделение объектов приложения. Схема включает таблицы, которые можно сгруппировать по функциональным доменам:

Группа 1: Управление пользователями (4 таблицы)

- user_info — базовые учётные данные
- user_params — физические параметры и цели
- user_weight — история изменений веса
- user_refresh_token — управление сессиями

Группа 2: Тренировки (3 таблицы)

- exercise — справочник упражнений
- workout — тренировочные сессии
- workout_exercise — связь тренировок и упражнений

Группа 3: Питание и мониторинг (3 таблицы)

- user_food — дневник питания
- user_calories — журнал калорий
- user_recommendation — история рекомендаций

Группа 4: Инфраструктура (3 таблицы)

- tasks — очередь фоновых задач

Рис. 9. Схема БД сервиса

2.5.3. Модель пользователей и профиля

Основу модели пользователей составляет таблица user_info, содержащая базовые учётные данные. Её центральным полем является id типа UUID, выбранный в качестве первичного ключа вместо традиционного serial. Это решение обусловлено тремя ключевыми преимуществами: глобальной уникальностью, важной для потенциально распределённых сценариев; безопасностью, поскольку UUID невозможно подобрать последовательно; отсутствием конфликтов при миграции данных между различными инстансами базы данных. Поле username типа TEXT с ограничением NOT NULL и уникальным индексом idx_user_info_username служит логином пользователя для аутентификации в сочетании с полем password, где хранится хэш пароля по алгоритму bcrypt с work factor 12 — такое хранение в хэшированном виде является обязательным требованием безопасности.

Поля nameи surname хранят персональные данные для персонализации интерфейса и не являются обязательными для минимизации барьера входа при регистрации. Поле email с уникальным ограничением используется для восстановления пароля и отправки уведомлений, при этом допускает значение NULL для пользователей, регистрирующихся исключительно по телефону, который хранится в поле phone и валидируется на уровне приложения. Метка времени created_at типа TIMESTAMPTZ с дефолтным значением NOW() фиксирует момент создания учётной записи и используется для аналитики пользовательской активности и расчёта retention rate.

Детальная информация о физических параметрах и целях пользователя вынесена в отдельную таблицу user_params, связанную с user_info отношением один-к-одному через внешний ключ id_user. Это разделение обусловлено принципом единой ответственности — user_infoотвечает за аутентификацию, user_params — за бизнес-данные профиля.
Поле height хранит рост в сантиметрах и используется в формуле Миффлина-Сан Жеора для расчёта базового метаболизма (BMR), допуская NULL для пользователей, не указавших этот параметр. Поле photo содержит ссылку на аватар в S3-хранилище, храня полный URL вместо только ключа объекта для упрощения клиентских приложений.

Ключевые для алгоритмов системы поля wants и lifestyle используют типы ENUM: wants определяет фитнес-цель пользователя (похудеть, набрать массу или поддерживать форму), lifestyle — уровень повседневной активности (сидячий, умеренно активный, очень активный) и выступает множителем в формуле TDEE (Total Daily Energy Expenditure).
Поле target_workouts_weeks задаёт целевое количество тренировок в неделю для планировщика, target_calories_daily — суточную норму калорий, рассчитываемую автоматически

на основе BMR с учётом уровня активности и цели, target_weight — целевой вес в килограммах для отслеживания прогресса.

Динамика изменений веса пользователя хранится в таблице user_weight, реализующей паттерн временных рядов. Выделение истории веса в отдельную таблицу вместо хранения массива значений в user_params продиктовано требованиями нормализации данных, эффективности запросов и масштабируемости. Поле weight типа FLOAT хранит вес в килограммах с точностью до 0.1 кг, использование FLOAT вместо DECIMAL обосновано упрощением вычислений и экономией места при сохранении достаточной точности для медицинских целей. Поле date типа TIMESTAMPTZ с дефолтом NOW() фиксирует момент взвешивания и в сочетании с составным индексом (id_user, date) позволяет эффективно строить графики прогресса и вычислять среднюю скорость изменения веса за произвольные периоды.

Управление сессиями аутентификации осуществляется через таблицу user_refresh_token, где поле refresh_token хранит хэшированный по алгоритму SHA256 refresh-токен. Хэширование критически важно для безопасности, поскольку refresh-токены имеют длительный срок жизни (30 дней), и при компрометации базы данных злоумышленник не сможет использовать украденные токены для получения новых access-токенов. Связь с пользователем через user_id обеспечивает каскадное удаление токенов при удалении учётной записи.

2.5.4. Модель тренировок и упражнений

Централизованный каталог упражнений представлен таблицей exercise, содержащей детальные метаданные для алгоритмов генерации тренировок. Поле level_preparation типа ENUM определяет требуемый уровень подготовки (новичок, средний, спортсмен) и индексируется для ускорения подбора упражнений под конкретного пользователя. Поле nameтипа VARCHAR(100) ограничивает длину названия упражнения 100 символами, балансируя между гибкостью TEXT и контролем длины VARCHAR, с индексом idx_exercise_name для поиска по названию.
Поле type_exercise (кардио, силовые, на гибкость) также индексируется для фильтрации по типу.

Технические параметры включают base_count_reps — базовое количество повторений для среднего уровня, корректируемое алгоритмом под уровень пользователя; steps — количество подходов или шагов; link_gif — ссылку на анимацию с демонстрацией техники, хранящуюся в S3. Поле place_exercise определяет место выполнения (дом, зал, улица) для подбора упражнений под доступное оборудование. Поле avg_calories_per типа DECIMAL(5,2) хранит средние затраты калорий за одно повторение или минуту с точностью до 0.01 ккал в диапазоне 0-999.99,

использование DECIMAL вместо FLOAT обусловлено требованиями точности, аналогичными финансовым вычислениям. Поле base_relax_time задаёт базовое время отдыха между подходами в секундах с последующей корректировкой под уровень пользователя и интенсивность тренировки.

Тренировочные сессии хранятся в таблице workout, где поле user_id связывает тренировку с пользователем через внешний ключ с индексом для быстрого поиска. Поле level определяет уровень сложности тренировки (лёгкая, средняя, тяжёлая) и используется в алгоритме прогрессивной нагрузки — при успешном выполнении серии тренировок уровень повышается. Поле status отражает жизненный цикл тренировки: создана, выполняется, завершена или провалена, с составным индексом (user_id, status) для эффективного поиска активных тренировок пользователя.

Поля total_calories и prediction_calories хранят соответственно фактические и прогнозируемые затраты калорий, их сравнение используется для калибровки алгоритмов и анализа эффективности тренировок. Поле duration типа INTERVAL хранит планируемую продолжительность тренировки, использование INTERVAL вместо INTEGER секунд позволяет естественно работать с временными промежутками в SQL-запросах.
Поля created_at и updated_at фиксируют время создания и последнего обновления с соответствующими индексами для сортировки и контроля интервалов между тренировками (минимальный интервал 8 часов).

Связь многие-ко-многим между тренировками и упражнениями реализована через таблицу workout_exercise, которая помимо внешних ключей содержит поле status для отслеживания выполнения каждого упражнения в реальном времени (не начато, выполняется, завершено, пропущено). Такое разделение позволяет избежать дублирования данных атрибутов упражнений и обеспечивает гибкость в отслеживании прогресса выполнения тренировки.

2.5.5. Модель питания и мониторинга

Дневник питания реализован в таблице user_food, где каждому приёму пищи соответствует запись с детализацией пищевой ценности. Поля protein, carbs, fat хранят содержание белков, углеводов и жиров в граммах соответственно и используются для расчёта баланса БЖУ.
Поле calorier содержит калорийность в килокалориях, являясь расчётным значением по формуле: калории = белки×4 + углеводы×4 + жиры×9. Поле description хранит описание блюда, заполняемое либо AI при анализе фотографии, либо вручную пользователем. Поле datetипа TIMESTAMPTZ с

индексом позволяет агрегировать данные за день, неделю или месяц для сравнения с целевыми показателями из user_params. Интеграция с Open AI API обеспечивает автоматическое заполнение полей БЖУ при анализе фотографий еды, при этом сохраняется возможность ручного ввода и корректировки.

Дополнительный журнал калорий в таблице user_calories служит для учёта калорий вне детального дневника питания. Поле calories типа INT может содержать как положительные значения (потребление), так и отрицательные (затраты), что позволяет учитывать физическую активность, не связанную с тренировками системы (ходьба, плавание), произвольные перекусы без детального учёта БЖУ и корректировки расхода калорий. Это обеспечивает дополнительный канал данных для более полной картины энергетического баланса.

История персонализированных рекомендаций хранится в таблице user_recommendation, где поле description содержит текст совета, генерируемого AI на основе комплексного анализа данных пользователя.

Ежедневная фоновая задача анализирует дефицит или профицит калорий за последние 7 дней, прогресс в тренировках, динамику веса и цели пользователя для формирования релевантных рекомендаций по питанию, тренировкам, восстановлению и мотивации. Индексы по user_id и date обеспечивают эффективный доступ к истории рекомендаций для каждого пользователя.

2.5.6. Модель фоновых задач

Очередь отложенных задач реализована в таблице **tasks**, обеспечивающей надёжность выполнения асинхронных операций. Поле **task_type** определяет тип задачи: генерация тренировки, отправка уведомления, анализ фотографии еды, генерация рекомендации. Поле **task_state** отражает текущее состояние задачи (ожидает, выполняется, завершена, провалена). Механизм повторных попыток реализован через поля **max_attempts** (максимальное количество попыток, по умолчанию 3), **attempts** (текущее количество выполненных попыток) и **retry_at** (время следующей попытки, вычисляемое по формуле экспоненциальной задержки: NOW() + 2^attempts × 60 секунд).
Поле **attribute** типа JSONB хранит сериализованные атрибуты задачи в структурированном формате, например, для уведомления: {"type": "email", "subject": "...", "body": "..."}.

Использование JSONB вместо отдельной таблицы атрибутов обеспечивает гибкость хранения произвольных параметров без изменения схемы базы данных.

Такая архитектура гарантирует, что задачи не теряются при перезапуске сервера и могут обрабатываться несколькими воркерами параллельно.

2.5.7. Миграции и управление схемой

Управление эволюцией схемы базы данных организовано через систему миграций на основе SQL-файлов, расположенных в директории migrations с чёткой нумерацией и разделением на файлы up (применение) и down (откат).

Каждая миграция проектируется с соблюдением принципа идемпотентности — возможности многократного применения без побочных эффектов, что достигается использованием конструкций типа CREATE TABLE IF NOT EXISTS и ALTER TABLE ... ADD COLUMN IF NOT EXISTS. Обратимость обеспечивается наличием для каждой миграции up соответствующей миграции down, полностью отменяющей изменения.

Для минимизации влияния на работоспособность системы миграции выполняются с минимальным блокированием, где это возможно, например, создание индексов с указанием CONCURRENTLY. Валидация данных интегрирована в миграции через проверочные блоки DO, которые перед добавлением ограничений проверяют соответствие существующих данных новым требованиям.

Процесс внедрения изменений включает пять этапов: разработка миграций в feature-ветке, тестирование на тестовой БД с проверкой производительности, ревью SQL-кода на безопасность и оптимальность, применение в production во время низкой нагрузки и мониторинг производительности после миграции.

Версионирование через timestamp в названиях файлов гарантирует корректный порядок применения миграций, а система поддерживает откат к любой предыдущей версии схемы при необходимости.

Выводы по главе

В данной главе были рассмотрены ключевые архитектурные решения, выбранные для проектирования системы, используемые технологии разработки серверной части, а также подходы к организации хранения и обмена данными. Были приведены обоснования применения микросервисной архитектуры, API Gateway, событийно-ориентированного взаимодействия, использования JWT-аутентификации, CQRS для высоконагруженных компонентов и выбора конкретных технологических стеков для отдельных сервисов.

Проектируемая платформа ориентирована на работу с большим количеством пользователей, активностей и взаимосвязанных данных, а также на устойчивую обработку потоков событий и частых операций чтения. В связи с этим особое внимание было уделено масштабируемости, отказоустойчивости и разделению ответственности между компонентами системы. Выбор микросервисного подхода позволяет независимо развивать и масштабировать отдельные домены, а применение брокера сообщений снижает связанность сервисов и повышает стабильность при пиковых нагрузках. Использование API Gateway и централизованного контура безопасности упрощает управление доступом и защищает внешний периметр системы.

Отдельное внимание уделено организации хранения данных: реляционные базы данных применяются для критичных структурированных сущностей, in-memory хранилища — для ускорения чтения и кеширования, объектное хранилище — для работы с медиа-контентом. Такое разделение позволяет эффективно использовать ресурсы и адаптироваться к росту объёмов данных. Выбор Kotlin и Spring-экосистемы обеспечивает баланс между производительностью, безопасностью, скоростью разработки и удобством сопровождения.

В совокупности принятые архитектурные и технологические решения формируют гибкую, расширяемую и устойчивую платформу, способную корректно функционировать при увеличении числа пользователей, объёмов данных и сложности бизнес-логики, а также обеспечивать дальнейшее развитие системы без существенных архитектурных переработок.

Глава 3. Программная реализация серверной части

В данной главе описывается практическая реализация серверной части платформы персонализированного контроля питания и тренировок. На основе архитектурных решений, обоснованных в Главе 2, и требований, формализованных в Техническом задании, были разработаны все ключевые модули системы. Реализация выполнена на языке программирования Go с использованием фреймворка Gin для HTTP-обработки, PostgreSQL в качестве основной базы данных, Redis для кэширования и MinIO для объектного хранения. В главе последовательно рассматриваются структура проекта, реализация каждого модуля и их интеграция в единую систему.

3.1. Структура проекта и организация кода

В соответствии с архитектурными решениями, принятыми на этапе проектирования, серверная часть платформы реализована как модульный монолит с четким разделением на слои согласно принципам Clean Architecture и выделением доменных контекстов в рамках Domain-Driven Design. Такая организация кодовой базы обеспечивает высокую связность внутри доменных модулей и низкую связанность между слоями, что критически важно для тестируемости, поддерживаемости и возможности дальнейшего развития системы.

Корневая структура проекта организована следующим образом. Каталог cmd содержит точки входа в приложение, где происходит инициализация всех компонентов и запуск HTTP-сервера. Каталог internal включает основной код приложения, недоступный для импорта извне, что соответствует идиоматике языка Go и подчеркивает инкапсуляцию внутренней логики. Внутри internal выделены пакеты domain для бизнес-сущностей, service для сценариев использования, infrastructure для реализаций репозиториев и внешних интеграций, handlers для HTTP-обработчиков, dto для объектов передачи данных и app для связывания всех компонентов.

Особого внимания заслуживает организация пакета service, где каждый подкаталог соответствует выделенному доменному контексту: auth, workouts, nutrition, recommendation, executor. Такое разделение обеспечивает логическую независимость модулей: изменения в логике тренировок минимально затрагивают код питания, а четкие интерфейсы между слоями позволяют легко заменять реализации, например, переходить с одного провайдера уведомлений на другой без изменения бизнес-логики.

Каталог migrations содержит версионированные SQL-скрипты для инициализации и обновления схемы базы данных. Каждая миграция имеет парные файлы up и down, что

обеспечивает возможность отката изменений при необходимости. Такой подход гарантирует согласованность структуры базы данных на всех окружениях — от разработки до продакшна.

3.2. Реализация модуля аутентификации (Auth Service)

Модуль аутентификации является точкой входа для всех пользователей системы и отвечает за регистрацию, вход в систему, управление сессиями и восстановление доступа. Реализация модуля базируется на требованиях технического задания и использует спроектированные таблицы базы данных для хранения учетной информации и управления токенами.

Процесс регистрации реализован в эндпоинте POST /auth/register. При поступлении запроса с данными пользователя, включающими username, password, email, имя, фамилию и телефон, сервис выполняет несколько последовательных шагов. На первом этапе производится валидация входных данных: проверяется уникальность username и email путем запросов к репозиторию пользователей. Это предотвращает дублирование учетных записей и гарантирует, что каждый пользователь идентифицируется уникальным логином. На втором этапе пароль пользователя хешируется с использованием алгоритма bcrypt, что обеспечивает надежное хранение учетных данных даже в случае компрометации базы данных. Затем создается запись в таблице user_info с предоставленными данными и хешем пароля. После успешного создания пользователя генерируется пара JWT-токенов: короткоживущий access-токен для доступа к защищенным ресурсам и долгоживущий refresh-токен для обновления сессии. Refresh-токен сохраняется в отдельной таблице для последующего управления сессиями, при этом в базе данных хранится его хешированная версия, что исключает возможность использования токена даже при прямом доступе к данным. В случае успеха клиенту возвращается пара токенов, а также базовая информация о пользователе.

Эндпоинт POST /auth/login обрабатывает запросы на вход в систему. Сервис по username находит пользователя в базе данных и сравнивает хеш пароля из БД с предоставленным в запросе. Для сравнения используется функция bcrypt, которая учитывает соль и позволяет проверить соответствие без раскрытия исходного пароля. При успешной аутентификации генерируется новая пара токенов, при этом старые refresh-токены пользователя инвалидируются, что предотвращает возможность использования украденных токенов. Затем новая пара возвращается клиенту.

Для защиты эндпоинтов от неавторизованного доступа реализован middleware на базе фреймворка Gin. Middleware извлекает токен из заголовка Authorization, валидирует его подпись и срок действия с использованием секретного ключа. В случае успеха информация о пользователе, а именно его идентификатор, сохраняется в контексте запроса, что позволяет всем последующим обработчикам получать текущего пользователя без необходимости повторного парсинга токена.

Если токен невалиден или отсутствует, middleware прерывает выполнение запроса и возвращает клиенту статус 401 Unauthorized. Такой подход применяется для всех защищенных маршрутов, включая работу с тренировками, питанием и профилем.

Механизм обновления токенов реализован через эндпоинт /auth/refresh. Клиент отправляет действующий refresh-токен, который сервис проверяет на существование в соответствующей таблице и на срок действия. Важно отметить, что refresh-токен используется только один раз: после успешной проверки он удаляется из базы данных, и генерируется новая пара токенов. Это предотвращает атаки повторного использования и повышает общую безопасность системы. При истечении срока действия refresh-токена пользователь должен пройти повторную аутентификацию.

3.3. Реализация модуля управления данными (CRUD Service)

Модуль управления данными является фундаментальным слоем инфраструктуры, предоставляющим базовые операции для работы со всеми ключевыми сущностями системы. В соответствии с техническим заданием и спецификацией API, данный модуль обеспечивает управление справочником упражнений, пользовательской информацией, параметрами пользователя и историей веса.

Базовым паттерном для реализации операций выбран паттерн Репозиторий. Для каждой основной сущности создан отдельный интерфейс в слое домена, а его конкретная реализация находится в слое инфраструктуры и использует библиотеку для взаимодействия с PostgreSQL с применением параметризованных запросов, защищающих от SQL-инъекций.

Управление пользовательскими данными разделено на несколько логических блоков. Основная информация пользователя управляется через эндпоинты группы /user/info. При получении запроса GET /user/info сервис извлекает идентификатор пользователя из контекста, установленный JWT middleware, и выполняет запрос к репозиторию для получения данных. Возвращаемая информация включает username, email, имя, фамилию, телефон и дату создания аккаунта. Обновление данных через PATCH /user/info реализовано с поддержкой частичного обновления: клиент может отправить только те поля, которые необходимо изменить. При обновлении email или телефона выполняется дополнительная валидация форматов, а также проверка уникальности, если эти данные уже используются другим пользователем.

Параметры пользователя, доступные по эндпоинтам /user/params, включают физические характеристики и фитнес-цели. Особенностью реализации является использование строго типизированных значений для полей, определяющих цель пользователя и уровень активности. При создании параметров через POST /user/params система автоматически рассчитывает суточную

норму калорий на основе формулы Миффлина-Сан Жеора. Расчет учитывает базовый метаболизм, умноженный на коэффициент активности, соответствующий выбранному образу жизни, а затем корректируется в зависимости от цели пользователя: при похудении создается дефицит калорий, при наборе массы — профицит. Рассчитанное значение сохраняется и используется в дальнейшем для сравнения с фактическим потреблением.

История веса реализована как временной ряд изменений и управляется через эндпоинты группы /user/weight. Эндпоинт POST /user/weight создает новую запись с текущим весом и временной меткой. Эндпоинт GET /user/weight возвращает последнюю запись, для чего используется SQL-запрос с сортировкой по дате и ограничением в одну запись. Эндпоинт GET /user/weight/history возвращает полную историю изменений, что позволяет клиентскому приложению строить графики прогресса и анализировать динамику изменения веса. При необходимости пользователь может удалить конкретную запись через DELETE /user/weight/{uuid}, например, если запись была ошибочной.

Справочник упражнений является ключевым для модуля тренировок. API предоставляет полный набор операций: создание нового упражнения через POST /exercises, получение списка с фильтрацией через GET /exercises, получение конкретного упражнения по ID, обновление через PATCH /exercises/{uuid} и удаление через DELETE /exercises/{uuid}. Особый интерес представляет фильтрация: клиент может запросить упражнения по уровню подготовки, типу упражнения и месту выполнения. Реализация фильтрации использует динамическое построение условий на основе переданных параметров, что позволяет эффективно выбирать только необходимые данные без дублирования кода. Использование строго типизированных значений на уровне базы данных и API обеспечивает целостность данных и исключает возможность сохранения некорректных значений.

3.4. Реализация модуля тренировок (Workout Service)

Модуль тренировок является одним из ключевых компонентов платформы, реализующим автоматическую генерацию персонализированных тренировочных программ. В отличие от простых CRUD-операций, этот модуль содержит сложную бизнес-логику, включающую анализ пользовательских данных, алгоритмы подбора упражнений, расчет энергозатрат и фоновую генерацию тренировок. Модуль предоставляет как публичные эндпоинты для генерации тренировок по запросу пользователя, так и фоновый воркер, автоматически создающий тренировки на основе расписания и анализа активности.

Архитектура модуля построена вокруг сервиса, который инкапсулирует всю логику генерации и управления тренировками. Сервис взаимодействует с несколькими репозиториями: для

получения параметров пользователя, его персональной информации, истории тренировок, справочника упражнений, а также для сохранения созданных тренировок и связи тренировок с упражнениями. Отдельно реализован менеджер транзакций, обеспечивающий целостность данных при создании тренировки вместе со списком упражнений.

3.4.1. Фоновая генерация тренировок

Ключевой особенностью модуля является фоновый воркер, который периодически запускается и анализирует необходимость генерации новых тренировок для всех активных пользователей. При запуске сервис инициализирует несколько горутин: основной цикл генерации, сборщик метрик и обработчик паник для обеспечения отказоустойчивости.

Основной цикл работает с заданным интервалом, который настраивается через конфигурацию сервиса. При каждом срабатывании воркер запускает процесс обработки всех пользователей, который выполняется с учетом таймаутов и ограничений по времени. Процесс организован постранично: пользователи загружаются батчами определенного размера, что предотвращает перегрузку базы данных и позволяет обрабатывать большое количество пользователей без исчерпания ресурсов.

Для каждого батча пользователей создается errgroup с ограничением на максимальное количество одновременно обрабатываемых пользователей. Это позволяет эффективно использовать вычислительные ресурсы, не создавая избыточной нагрузки на базу данных. Каждый пользователь обрабатывается в отдельной горутине с собственным контекстом и таймаутом, что обеспечивает изоляцию ошибок: сбой при обработке одного пользователя не влияет на остальных.

3.4.2. Анализ необходимости генерации

Перед генерацией тренировки для конкретного пользователя воркер выполняет комплексный анализ, чтобы определить, нужна ли новая тренировка в данный момент. Этот анализ учитывает множество факторов и реализован в методе analyzeWorkoutStats.

Первым шагом анализируется история последних тренировок пользователя. Загружаются последние тренировки в количестве, достаточном для принятия решения. На основе этой истории вычисляется, сколько тренировок пользователь выполнил за последнюю неделю, и сравнивается с целевым количеством, указанным в параметрах пользователя. Если недельная норма уже достигнута, генерация пропускается.

Далее проверяется статус последней тренировки. Если у пользователя есть активная незавершенная тренировка, генерация новой также пропускается — система считает, что

пользователь должен сначала завершить начатое. Это предотвращает накопление большого количества незавершенных тренировок и стимулирует пользователя доводить дела до конца.

Проверяется также количество созданных, но еще не начатых тренировок. Если их накопилось больше допустимого лимита, генерация откладывается. Это ограничение предотвращает ситуацию, когда у пользователя скапливается множество несделанных тренировок, что может демотивировать.

Важным фактором является время отдыха между тренировками. Система проверяет, сколько времени прошло с момента последней тренировки, и если это время меньше минимально допустимого интервала, установленного в восемь часов, генерация пропускается. Это соответствует физиологическим принципам восстановления и предотвращает перетренированность.

Если все проверки пройдены успешно, система переходит к следующему этапу — определению предпочтений пользователя. На основе истории тренировок анализируется, какие типы упражнений пользователь выполняет чаще всего и какие места для тренировок предпочитает. Эти данные будут использованы при подборе упражнений для новой тренировки.

Также на основе истории успешных выполнений определяется предпочтительный уровень сложности. Система анализирует, тренировки какого уровня пользователь завершает с наибольшим процентом успеха, и стремится генерировать тренировки именно этого уровня, постепенно повышая сложность при хороших результатах.

3.4.3. Процесс генерации тренировки

Когда решение о генерации принято, запускается основной алгоритм создания тренировки, реализованный с поддержкой повторных попыток. Учитывая, что процесс может завершиться неудачей, например, из-за временного отсутствия подходящих упражнений, реализован механизм повторных попыток с экспоненциальной задержкой. При первой неудаче система ждет 100 миллисекунд, затем 200, 400 и так далее, что позволяет справляться с временными сбоями.

На первом этапе генерации определяется целевой уровень сложности. Для этого уровень активности пользователя преобразуется в соответствующий уровень подготовки. Например, малоподвижному образу жизни соответствует начальный уровень, активному — средний, а спортивному — продвинутый.

Затем выполняется запрос к репозиторию упражнений для получения списка подходящих упражнений. Фильтрация происходит по уровню подготовки и предпочтительному месту тренировок. Если по заданным критериям упражнений не найдено, фильтр по месту снимается, и

поиск выполняется без этого ограничения. Это гарантирует, что пользователь всегда получит тренировку, даже если для его любимого места временно нет подходящих упражнений в базе.

Следующий этап — выбор конкретных упражнений для тренировки. Количество упражнений выбирается случайным образом в заданном диапазоне, обычно от четырех до двенадцати. Алгоритм выбора учитывает предпочтения пользователя: около шестидесяти процентов упражнений будут того типа, который пользователь предпочитает, а остальные — других типов для обеспечения разнообразия и сбалансированной нагрузки. Упражнения случайным образом перемешиваются, чтобы каждая тренировка была уникальной.

После выбора упражнений рассчитываются прогнозируемые параметры тренировки. Для каждого упражнения вычисляются ожидаемые затраты калорий и длительность выполнения с учетом коэффициента, зависящего от уровня подготовки пользователя. Базовые значения корректируются: для продвинутого уровня калории и длительность увеличиваются, для начального — остаются базовыми. Затем рассчитывается общее время с учетом отдыха между упражнениями, который составляет одну минуту.

Финальный этап — сохранение тренировки в базе данных. Эта операция выполняется в рамках транзакции, чтобы гарантировать целостность данных. Сначала создается запись в таблице тренировок с метаданными: идентификатор пользователя, уровень сложности, статус «создана», прогнозируемые калории и длительность. Затем для каждого выбранного упражнения создается связующая запись в таблице workout_exercise с указанием порядка выполнения, базовых повторений, времени отдыха и статуса «ожидает». Если любая из операций завершается ошибкой, транзакция откатывается, и пользователь не получает некорректную тренировку.

После успешного сохранения тренировка возвращается вызывающему коду. Если генерация происходила в фоновом воркере и включены уведомления, создается асинхронная задача на отправку push-уведомления пользователю о том, что для него сгенерирована новая персональная тренировка.

3.4.4. Генерация тренировки по запросу

Помимо фоновой генерации, модуль предоставляет публичный эндпоинт для создания тренировки по параметрам, указанным пользователем. Этот метод, GenerateCustomWorkout, позволяет пользователю самостоятельно задать параметры: количество упражнений, уровень сложности, место тренировки и тип упражнений.

Алгоритм работы аналогичен фоновой генерации, но с некоторыми отличиями. Во-первых, уровень сложности, указанный пользователем, преобразуется в коэффициент, который применяется ко всем расчетам: для легкого уровня коэффициент равен единице, для среднего — полутора, для тяжелого — двум. Это позволяет создавать тренировки именно той интенсивности, которую запросил пользователь.

Во-вторых, фильтрация упражнений выполняется строго по указанным параметрам. Если пользователь выбрал конкретный тип упражнений, система будет выбирать упражнения только этого типа. Если же тип не указан, алгоритм старается сбалансировать тренировку, равномерно распределяя упражнения по доступным типам. Для этого упражнения группируются по типам, и из каждой группы выбирается примерно равное количество.

В-третьих, при пустом результате поиска система последовательно снимает фильтры: сначала по месту, затем по типу, расширяя поиск до тех пор, пока не будут найдены подходящие упражнения. Это гарантирует, что пользователь всегда получит тренировку, даже если его запрос был слишком специфичным.

3.4.5. Обработка ошибок и отказоустойчивость

Модуль спроектирован с учетом возможных сбоев и ошибок. Во всех критических точках реализована обработка паник с восстановлением и логированием стек-трейса. При панике в любой горутине сервис логирует ошибку и продолжает работу, не останавливая весь модуль.

Для операций с базой данных используются контексты с таймаутами, что предотвращает зависание при недоступности базы данных. При недоступности внешних сервисов или временных ошибках срабатывает механизм повторных попыток с экспоненциальной задержкой.

Все ошибки логируются с достаточным контекстом для диагностики: идентификатор пользователя, параметры запроса, этап выполнения. Это позволяет быстро выявлять и устранять проблемы в работе модуля.

Таким образом, модуль тренировок представляет собой сложную, но надежную систему, способную в автоматическом режиме обслуживать тысячи пользователей, генерируя для каждого персональные тренировки с учетом их целей, уровня подготовки и предпочтений, при этом соблюдая физиологические принципы восстановления и прогрессивной нагрузки.

3.5. Реализация модуля питания (Nutricion Service)

Модуль питания является ключевым компонентом платформы, реализующим интеллектуальный анализ потребляемой пищи с использованием технологий искусственного интеллекта. Модуль обеспечивает загрузку и анализ фотографий еды, ручной ввод данных, ведение дневника питания и формирование отчетов.

Центральной функцией модуля является анализ фотографий еды с использованием Open AI. Процесс анализа инициируется клиентским приложением, которое загружает фотографию в S3-хранилище через предварительно сгенерированный URL для прямой загрузки. После успешной загрузки клиент отправляет запрос на анализ, содержащий ссылку на изображение. Сервис питания формирует запрос к OpenAI с промптом, оптимизированным для определения пищевой ценности. Промпт запрашивает возврат структурированного JSON-объекта с описанием блюда, калорийностью и содержанием белков, жиров и углеводов. Важной особенностью промпта является указание делать консервативные оценки размера порций, что повышает точность результатов, особенно при фотографиях, где сложно определить реальный объем пищи.

Ответ от OpenAI парсится, и на его основе создается запись в дневнике питания. Запись содержит описание блюда, калории, белки, жиры, углеводы, ссылку на фотографию и временную метку. После сохранения записи система создает отложенную задачу для проверки баланса питания: анализируется суммарное потребление за день и при значительном отклонении от целевых показателей будет сгенерирована рекомендация. Например, если пользователь недобрал белка или превысил суточную норму калорий, система уведомит его об этом и предложит скорректировать рацион.

Помимо автоматического анализа, модуль поддерживает ручной ввод приемов пищи. Пользователь может создать запись с указанием названия блюда, количества и пищевой ценности, отредактировать существующую запись или удалить ошибочную. Все записи группируются по датам и типам приема пищи, таким как завтрак, обед, ужин или перекус, что позволяет получать структурированный дневник питания и анализировать пищевые привычки в разрезе времени суток.

Для анализа потребления реализован метод получения дневника питания за день. Метод выполняет запрос к репозиторию для получения всех записей пользователя за указанную дату, затем агрегирует данные, вычисляя суммарное потребление калорий и макронутриентов. Результат также содержит целевые показатели пользователя, полученные из таблицы параметров, и отклонение от цели. Это позволяет клиентскому приложению наглядно отображать прогресс, показывать цветовые индикаторы достижения целей и предупреждать пользователя о недостатке или избытке калорий еще до того, как это скажется на результате.

3.6. Реализация модуля рекомендаций (Recommendation Service)

Модуль рекомендаций отвечает за генерацию персонализированных советов по питанию, тренировкам и восстановлению на основе комплексного анализа данных пользователя. В отличие от других модулей, сервис рекомендаций не имеет публичных эндпоинтов для создания советов — все рекомендации генерируются фоновыми задачами, а для их получения используется эндпоинт GET /recommendations.

Модуль построен на основе комбинации правил и AI-генерации. Простые рекомендации генерируются по заранее определенным правилам, а сложные мотивационные советы и развернутые рекомендации создаются с использованием OpenAI API. Такой гибридный подход позволяет сочетать надежность и предсказуемость правил с гибкостью и креативностью искусственного интеллекта.

Система правил анализирует различные аспекты активности пользователя. Правило дефицита калорий срабатывает, если фактическое потребление за последние несколько дней ниже целевого более чем на определенный процент, и рекомендует увеличить калорийность рациона или добавить перекус. Правило баланса макронутриентов анализирует пропорции белков, жиров и углеводов: при недостатке белка генерируется рекомендация включить в рацион больше белковых продуктов, таких как мясо, рыба, яйца или бобовые. Правило регулярности тренировок сравнивает фактическое количество тренировок за неделю с целевым и при отклонении напоминает о необходимости соблюдать график, предлагая конкретные дни для ближайших занятий. Правило прогресса веса анализирует скорость изменения веса и при отклонении от целевой траектории корректирует рекомендации по питанию, например, советуя немного уменьшить порции, если вес снижается слишком медленно.

Для случаев, требующих более глубокого анализа или мотивационной поддержки, используется OpenAI API. При генерации AI-рекомендации формируется промпт, включающий агрегированные данные пользователя за последние семь дней: цель, уровень активности, среднее потребление калорий и белка, количество выполненных тренировок, изменение веса. На основе этих данных AI генерирует персонализированный совет на русском языке, учитывающий конкретную ситуацию пользователя. Например, если пользователь худеет, но при этом не выполняет силовые тренировки, AI может порекомендовать добавить их для сохранения мышечной массы, объяснив, почему это важно для долгосрочного результата.

Все сгенерированные рекомендации сохраняются в соответствующей таблице с указанием типа, приоритета, заголовка, описания и срока действия. Рекомендации с высоким приоритетом,

например, критический дефицит калорий или длительное отсутствие тренировок, также инициируют отправку уведомления через сервис уведомлений, чтобы привлечь внимание пользователя немедленно, а не только при следующем входе в приложение.

Эндпоинт GET /recommendations возвращает список активных рекомендаций для текущего пользователя, отсортированный по приоритету и дате создания. При получении рекомендации клиент может отметить ее как прочитанную через PATCH /recommendations/{uuid}/read, что позволяет системе отслеживать вовлеченность пользователя и не показывать одни и те же советы повторно. Эндпоинт POST /recommendations/refresh позволяет пользователю принудительно запросить генерацию новых рекомендаций, что полезно после существенных изменений в данных, например, после обновления цели или после длительного перерыва в тренировках.

3.7. Реализация модуля уведомлений (Executor Service)

Сервис уведомлений реализован как фоновый воркер, обеспечивающий надежную обработку отложенных задач. В соответствии со спроектированной структурой базы данных, сервис отвечает за отправку уведомлений по различным каналам, генерацию контента и выполнение периодических операций.

Таблица задач спроектирована для поддержки различных типов операций: отправка email, SMS и push-уведомлений, фоновая генерация тренировок, анализ фотографий еды, проверка баланса питания, генерация рекомендаций и напоминания о тренировках. Каждая задача имеет статус, количество попыток, максимальное количество попыток и время следующей попытки. Атрибуты задачи хранятся в JSON-поле, что обеспечивает гибкость хранения произвольных параметров без изменения схемы базы данных. Например, для задачи отправки email в атрибутах могут храниться адрес получателя, тема письма и его содержимое, а для задачи генерации тренировки — идентификатор пользователя и предпочтительные параметры.

Сервис уведомлений реализован как многопоточный воркер, способный обрабатывать несколько задач параллельно. При запуске сервис инициализирует пул воркеров, каждый из которых работает в отдельной горутине, что позволяет эффективно использовать ресурсы многоядерных процессоров. Диспетчер задач периодически опрашивает базу данных в поиске задач со статусом ожидания и временем следующей попытки, не превышающим текущее. Найденные задачи распределяются по воркерам в циклическом порядке для равномерной загрузки.

При получении задачи воркер переводит ее в статус выполнения и выполняет соответствующую операцию в зависимости от типа задачи. Для отправки email используется интеграция с SendGrid, обеспечивающая высокую надежность доставки и возможность

отслеживания статусов. Для отправки SMS используется Twilio с поддержкой большого количества стран и форматов номеров. Для push-уведомлений реализована интеграция с Firebase Cloud Messaging с Apple Push Notification Service для iOS.

Если операция завершается успешно, задача переводится в статус завершения. В случае ошибки воркер увеличивает счетчик попыток и, если количество попыток не превышает максимальное, переводит задачу обратно в статус ожидания с установкой времени следующей попытки по формуле экспоненциальной задержки. Это означает, что первая повторная попытка произойдет через одну минуту, вторая — через две минуты, третья — через четыре минуты и так далее. Такой подход позволяет справляться с временными сбоями внешних сервисов, например, кратковременной недоступностью API провайдера, не перегружая их частыми запросами. Если все попытки исчерпаны, задача переводится в статус ошибки с сохранением информации об ошибке для последующего анализа и ручного вмешательства при необходимости.

Помимо обработки отложенных задач, сервис уведомлений управляет периодическими задачами через встроенный планировщик. Ежедневно в восемь утра запускается генерация персонализированных рекомендаций для всех активных пользователей, чтобы они могли получить свежие советы к началу дня. В десять утра система проверяет, не пропустил ли пользователь запланированную тренировку, и при необходимости отправляет напоминание, учитывая, что многие планируют занятия на утренние часы. В двенадцать и восемнадцать часов отправляются напоминания о приеме пищи, соответствующие типичному времени обеда и ужина. Ночью, в три часа, запускается очистка старых завершенных задач для предотвращения неограниченного роста таблицы и поддержания производительности базы данных.

Для обеспечения наблюдаемости работы сервиса реализован сбор метрик через Prometheus. Отслеживаются количество задач по типам и статусам, время выполнения задач, количество повторных попыток, процент успешных и упавших задач, а также задержки в очереди. Эти метрики позволяют оперативно выявлять проблемы с доставкой уведомлений, например, если у какого-то провайдера возникли сбои, и оптимизировать работу сервиса, регулируя количество воркеров или интервалы между повторными попытками.

3.8. Реализация модуля аватаров (Avatar Service)

Модуль аватаров реализует специфичный для работы с файлами функционал, используя S3-совместимое объектное хранилище MinIO. Особенность реализации заключается в том, что файлы не передаются через сервер приложения, а загружаются напрямую клиентом в S3 по предварительно сгенерированной ссылке. Такой подход имеет несколько важных преимуществ:

сервер не тратит ресурсы на обработку больших файлов, уменьшается нагрузка на сетевое соединение, повышается скорость загрузки, а сам процесс может быть распараллелен и оптимизирован на стороне клиента.

Эндпоинт POST /avatars принимает запрос с указанием MIME-типа загружаемого файла и возвращает клиенту три значения: URL для прямой загрузки в S3, ключ объекта в хранилище и публичный URL для доступа к аватарке после загрузки. URL для загрузки действителен ограниченное время, обычно пятнадцать минут, что является дополнительной мерой безопасности, предотвращающей использование ссылки злоумышленниками после завершения сессии.

При генерации ссылки сервис создает уникальный ключ для объекта, включающий идентификатор пользователя и случайный UUID, что гарантирует отсутствие коллизий и предотвращает случайную или намеренную перезапись существующих файлов. Затем с использованием AWS SDK формируется подписанный URL для PUT-запроса, который подписывается учетными данными доступа к S3. Полученный URL возвращается клиенту, который может немедленно приступить к загрузке файла.

После успешной загрузки файла по предоставленному URL клиенту необходимо обновить ссылку на аватар в профиле пользователя. Это делается через эндпоинт PATCH /user/params с полем photo, содержащим полученный публичный URL. Таким образом, сервер приложения никогда не работает с бинарными данными изображений, а только с метаданными и ссылками, что существенно упрощает архитектуру и повышает безопасность, исключая возможность загрузки вредоносных файлов через сервер.

При обновлении аватара старая ссылка может быть удалена из профиля, но сам файл в S3 не удаляется немедленно — это позволяет реализовать откат изменений или восстановление в случае ошибки. Для очистки неиспользуемых файлов может быть реализована отдельная фоновая задача, анализирующая, какие изображения не привязаны ни к одному профилю.

Выводы по главе

В данной главе была представлена полная реализация серверной части платформы персонализированного фитнеса. На основе архитектурных решений, принятых в Главе 2, и требований, формализованных в Техническом задании, были разработаны и внедрены все ключевые модули системы.

Модуль аутентификации обеспечивает безопасный доступ с использованием JWT-токенов, хешированием паролей bcrypt и механизмом refresh-токенов с хранением в отдельной таблице, что соответствует современным стандартам безопасности.

Модуль управления данными предоставляет полный набор операций для работы со справочником упражнений, пользовательской информацией, параметрами и историей веса с поддержкой сложной фильтрации и пагинации, реализованной через паттерн Репозиторий.

Модуль тренировок реализует ключевую бизнес-логику платформы: интеллектуальную генерацию персонализированных тренировок с учетом места занятий, типа упражнений и уровня подготовки пользователя, а также отслеживание прогресса выполнения с детализацией по каждому упражнению.

Модуль питания интегрирован с OpenAI API для автоматического анализа фотографий еды и определения пищевой ценности, ведет детальный дневник питания с расчетом суточного баланса калорий и макронутриентов и сравнением с целевыми показателями.

Сервис уведомлений реализован как отказоустойчивый фоновый воркер с поддержкой очередей, повторных попыток с экспоненциальной задержкой, интеграцией с внешними провайдерами (SendGrid, Twilio, FCM/APNS) и планировщиком периодических задач для ежедневных напоминаний и генерации рекомендаций.

Модуль аватаров обеспечивает эффективную работу с медиафайлами через генерацию Presigned URL для прямой загрузки в S3-хранилище, полностью разгружая сервер приложения от обработки бинарных данных.

Все модули интегрированы в единую систему через четко определенные интерфейсы и следуют принципам Clean Architecture. Это обеспечивает тестируемость благодаря изоляции бизнес-логики от инфраструктурных деталей, масштабируемость через горизонтальное масштабирование Executor Service, отказоустойчивость благодаря механизмам повторных попыток и экспоненциальной задержки, а также наблюдаемость через сбор метрик Prometheus и логирование.

Разработанная серверная часть полностью готова к интеграции с клиентским приложением и способна обеспечить бесперебойную работу платформы для тысяч пользователей, автоматизируя полный цикл фитнес-сопровождения: от постановки целей и генерации персонализированных программ до отслеживания прогресса и мотивационной поддержки.

Заключение

В современном цифровом пространстве пользователи, стремящиеся к здоровому образу жизни, сталкиваются с необходимостью самостоятельного планирования тренировок, расчета калорий и отслеживания прогресса. Существующие фитнес-приложения часто предлагают стандартизированные решения, не учитывающие индивидуальные особенности пользователей, их цели и уровень подготовки. Пользователю требуется значительное количество времени и специальных знаний для того, чтобы сформировать эффективную программу тренировок и контролировать питание. В связи с этим возникает необходимость создания интеллектуальных инструментов, способных автоматизировать процесс планирования и предоставлять персонализированные рекомендации на основе анализа данных пользователя.

В рамках данной выпускной квалификационной работы была разработана серверная часть платформы BodyFuel, предназначенной для персонализированного контроля питания и тренировок с использованием технологий искусственного интеллекта. Особенностью платформы является автоматическая генерация тренировочных программ на основе анализа физических параметров пользователя, его целей и истории активности, а также интеллектуальный анализ питания с распознаванием блюд по фотографиям через интеграцию с OpenAI API. Такой подход упрощает процесс достижения фитнес-целей и помогает пользователям более эффективно управлять своим здоровьем без необходимости обращения к персональным тренерам и диетологам.

В ходе выполнения работы была проведена аналитика предметной области и рассмотрены существующие решения, такие как MyFitnessPal, FatSecret, Strong, Яндекс.Здоровье и 8fit. Проведенный сравнительный анализ по девяти ключевым критериям показал, что существующие платформы не предоставляют комплексного решения, объединяющего автоматическую генерацию тренировок, интеллектуальный анализ питания и динамическую адаптацию программ. Это подтвердило актуальность разработки платформы BodyFuel.

В рамках проектирования системы была разработана архитектура решения, основанная на принципах Clean Architecture и Domain-Driven Design с выделением доменных контекстов User, Workout, Nutrition и Recommendation. В архитектуру платформы входят сервис аутентификации, сервис тренировок, сервис питания, сервис рекомендаций, сервис уведомлений и модуль управления аватарами. Для реализации системы были выбраны современные технологии разработки серверных приложений, включая язык программирования Go, фреймворк Gin, систему управления базами данных PostgreSQL, систему кэширования Redis и объектное хранилище MinIO.

В процессе реализации были разработаны основные модули платформы. Модуль аутентификации отвечает за регистрацию и авторизацию пользователей с использованием JWT-токенов и хешированием паролей bcrypt. Модуль тренировок реализует автоматическую генерацию персонализированных программ с учетом предпочтений пользователя и принципа прогрессивной нагрузки, а также включает фоновый воркер для периодической генерации тренировок. Модуль питания обеспечивает анализ фотографий еды через OpenAI API и ведение детального дневника питания с расчетом суточного баланса калорий и макронутриентов. Модуль рекомендаций генерирует персонализированные советы на основе комплексного анализа данных пользователя. Сервис уведомлений, реализованный как отказоустойчивый фоновый воркер, обеспечивает отправку push-уведомлений, email и SMS с поддержкой повторных попыток и экспоненциальной задержки.

После завершения разработки было проведено тестирование системы в соответствии с документом «Программа и методика испытаний», включающее функциональное тестирование отдельных модулей, проверку интеграции между компонентами и тестирование производительности. Результаты тестирования показали корректность работы реализованного функционала, соответствие требованиям технического задания и стабильность взаимодействия компонентов системы.

Таким образом, поставленная цель работы — разработка серверной части интеллектуальной фитнес-платформы, обеспечивающей автоматическую генерацию персонализированных тренировочных программ и интеллектуальный анализ питания с использованием технологий искусственного интеллекта — была достигнута. В ходе работы были решены задачи анализа предметной области, сравнительного анализа существующих решений, определения требований к системе, проектирования архитектуры, разработки схемы базы данных, реализации всех ключевых сервисов и проведения тестирования разработанного программного обеспечения.

Разработанная платформа может быть в дальнейшем расширена и дополнена новыми возможностями. Среди возможных направлений развития системы можно выделить интеграцию с носимыми устройствами для автоматического сбора данных о физической активности, развитие алгоритмов машинного обучения для более точной генерации тренировок, внедрение социальных функций для взаимодействия пользователей, создание веб-версии приложения, а также повышение масштабируемости и отказоустойчивости системы за счет горизонтального масштабирования сервиса уведомлений и оптимизации работы с базой данных.

Список использованных источников

1. ГОСТ 19.101-77 Виды программ и программных документов. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77 Стадии разработки. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77 Обозначения программ и программных документов. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78 Основные надписи. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78 Общие требования к программным документам. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603-78 Общие правила внесения изменений. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001
9. REST — диссертация Р. Филдинга (архитектурный стиль) [Электронный ресурс]. – Режим доступа: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 26.01.2026).
10. MyFitnessPal — официальное приложение для отслеживания питания [Электронный ресурс]. – Режим доступа: https://apps.apple.com/us/app/myfitnesspal/id341232718 (дата обращения: 26.01.2026).
11. FatSecret — платформа для учёта калорий и питания [Электронный ресурс]. – Режим доступа: https://apps.apple.com/us/app/fatsecret-calorie-counter/id347184248 (дата обращения: 26.01.2026).
12. Strong — приложение для отслеживания силовых тренировок [Электронный ресурс]. – Режим доступа: https://apps.apple.com/us/app/strong-workout-tracker-gym-log/id464254577 (дата обращения: 26.01.2026).

13. Яндекс.Здоровье — сервис для отслеживания здоровья [Электронный ресурс]. – Режим доступа: https://yandex.ru/health/ (дата обращения: 26.01.2026).
14. 8fit — приложение для тренировок и питания [Электронный ресурс]. – Режим доступа: https://apps.apple.com/us/app/8fit-workouts-meal-plans/id874882792 (дата обращения: 26.01.2026).
15. Go — официальная документация языка программирования [Электронный ресурс]. – Режим доступа: https://go.dev/doc/ (дата обращения: 26.01.2026).
16. PostgreSQL — система управления базами данных [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 26.01.2026).
17. Redis — база данных типа ключ-значение [Электронный ресурс]. – Режим доступа: https://redis.io/documentation (дата обращения: 26.01.2026).
18. OpenAI Vision API — документация компьютерного зрения [Электронный ресурс]. – Режим доступа: https://platform.openai.com/docs/guides/vision (дата обращения: 26.01.2026).
19. Gin Web Framework — фреймворк для Go [Электронный ресурс]. – Режим доступа: https://gin-gonic.com/docs/ (дата обращения: 26.01.2026).
20. JWT (JSON Web Tokens) — стандарт аутентификации [Электронный ресурс]. – Режим доступа: https://jwt.io/introduction (дата обращения: 26.01.2026).
21. bcrypt — алгоритм хеширования паролей [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Bcrypt (дата обращения: 26.01.2026).
22. S3 API — спецификация объектного хранилища [Электронный ресурс]. – Режим доступа: https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html (дата обращения: 26.01.2026).
23. Clean Architecture — принципы чистой архитектуры [Электронный ресурс]. – Режим доступа: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (дата обращения: 26.01.2026).
24. Domain-Driven Design (DDD) — подход к проектированию [Электронный ресурс]. – Режим доступа: https://domainlanguage.com/ddd/ (дата обращения: 26.01.2026).
25. Mifflin-St Jeor Equation — формула расчёта базового метаболизма [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Basal_metabolic_rate (дата обращения: 26.01.2026).